



Orient BlackSwan

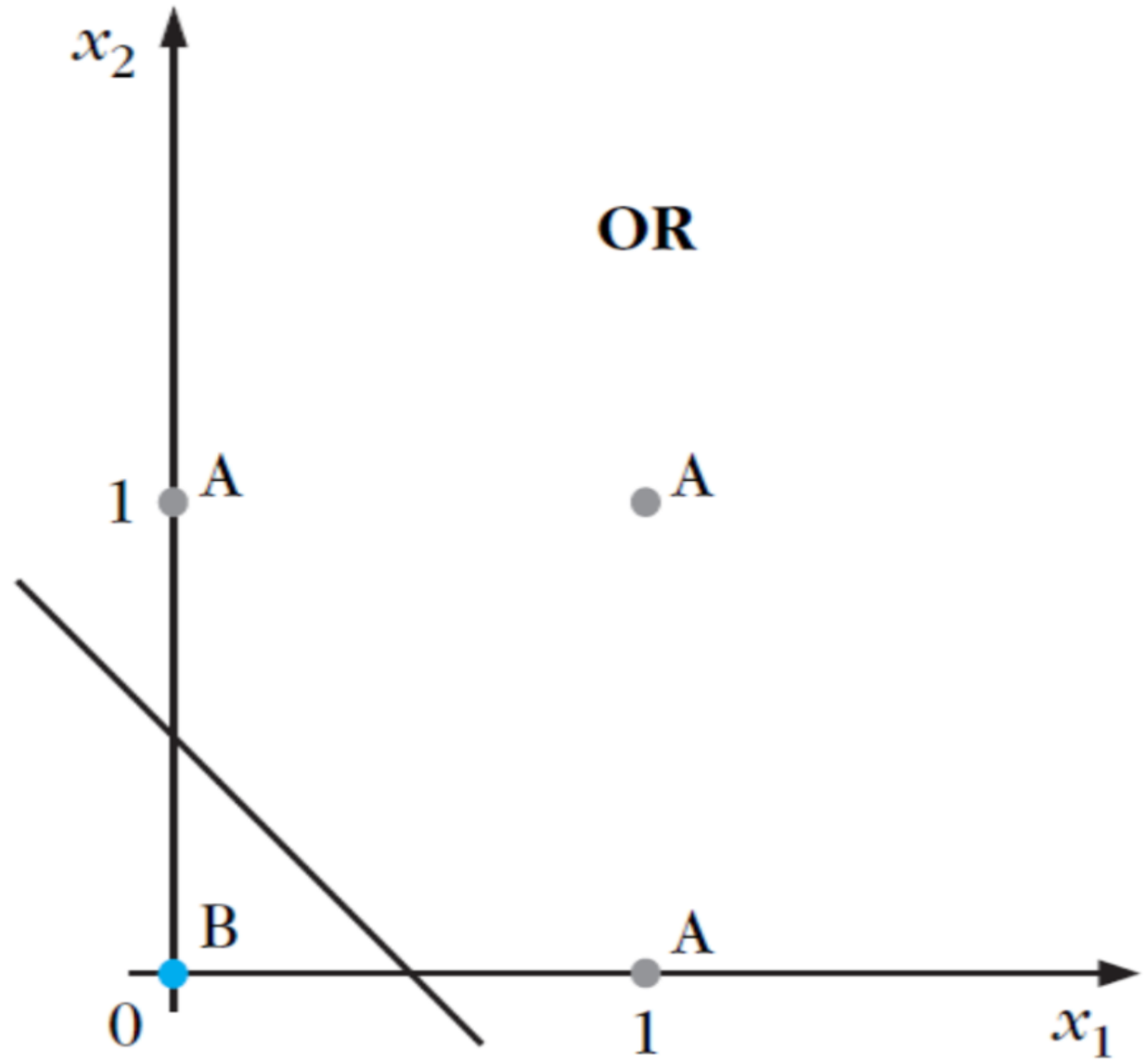
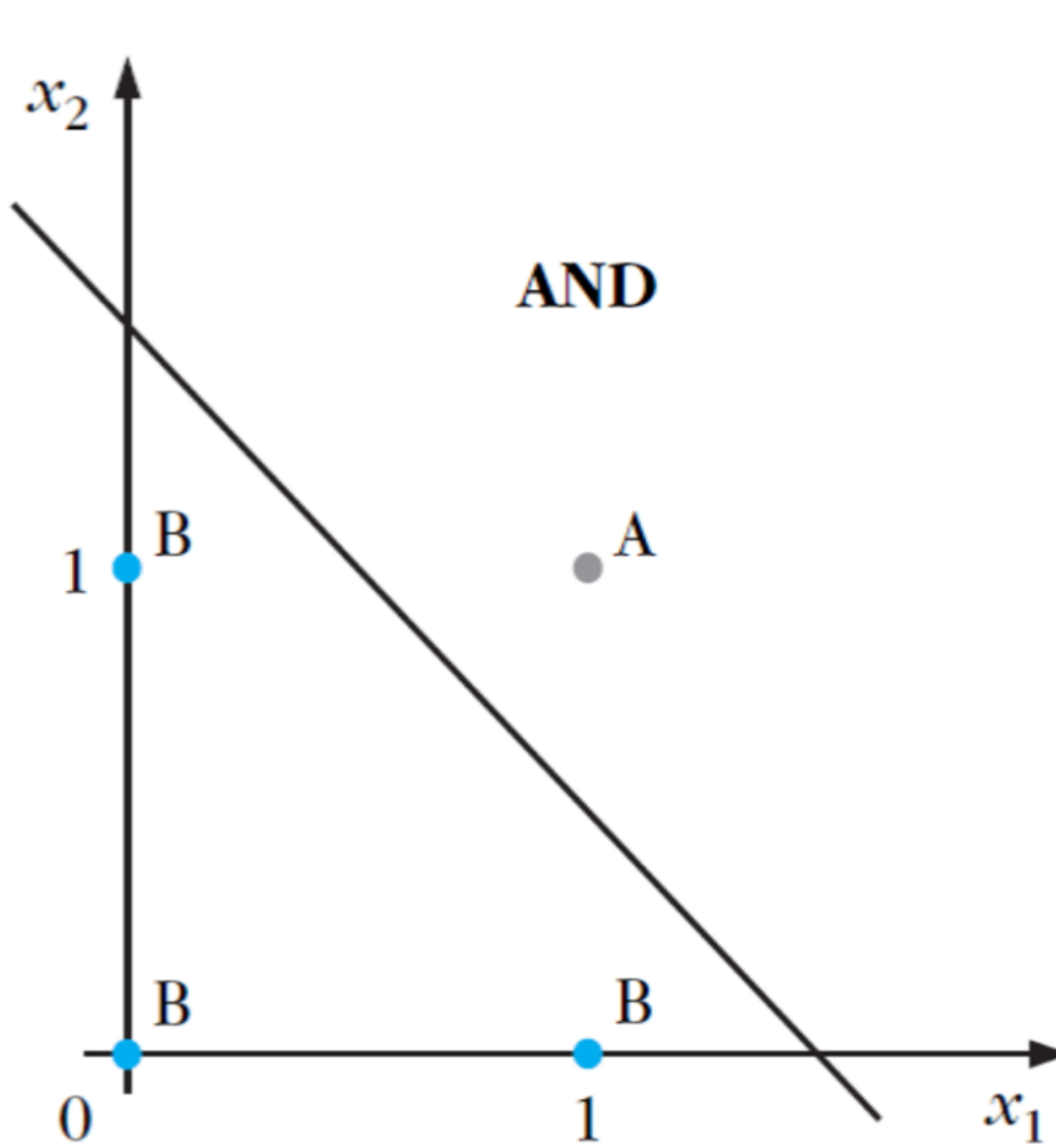
Linear Discriminants for ML

CHAPTER 8: MACHINE LEARNING – THEORY & PRACTICE

Linear Discriminant Functions

- $g(W,X) = W^t X + b = \sum_i w_i x_i + b$ is called a linear discriminant function.
- It is linear in the components of X , x_i and also in weights w_i .
- $g(W,X) = 0$ characterizes the decision boundary between the **positive** class ($g(W,X) > 0$) and the **negative** class ($g(W,X) < 0$).
- So, it can be used as a linear classifier.
- Note that $g(W,X) = W_a^t X_a$, where $X_a = (1, x_1, \dots, x_d)$ and $W_a = (b, w_1, \dots, w_d)$.

Boolean Functions: AND and OR

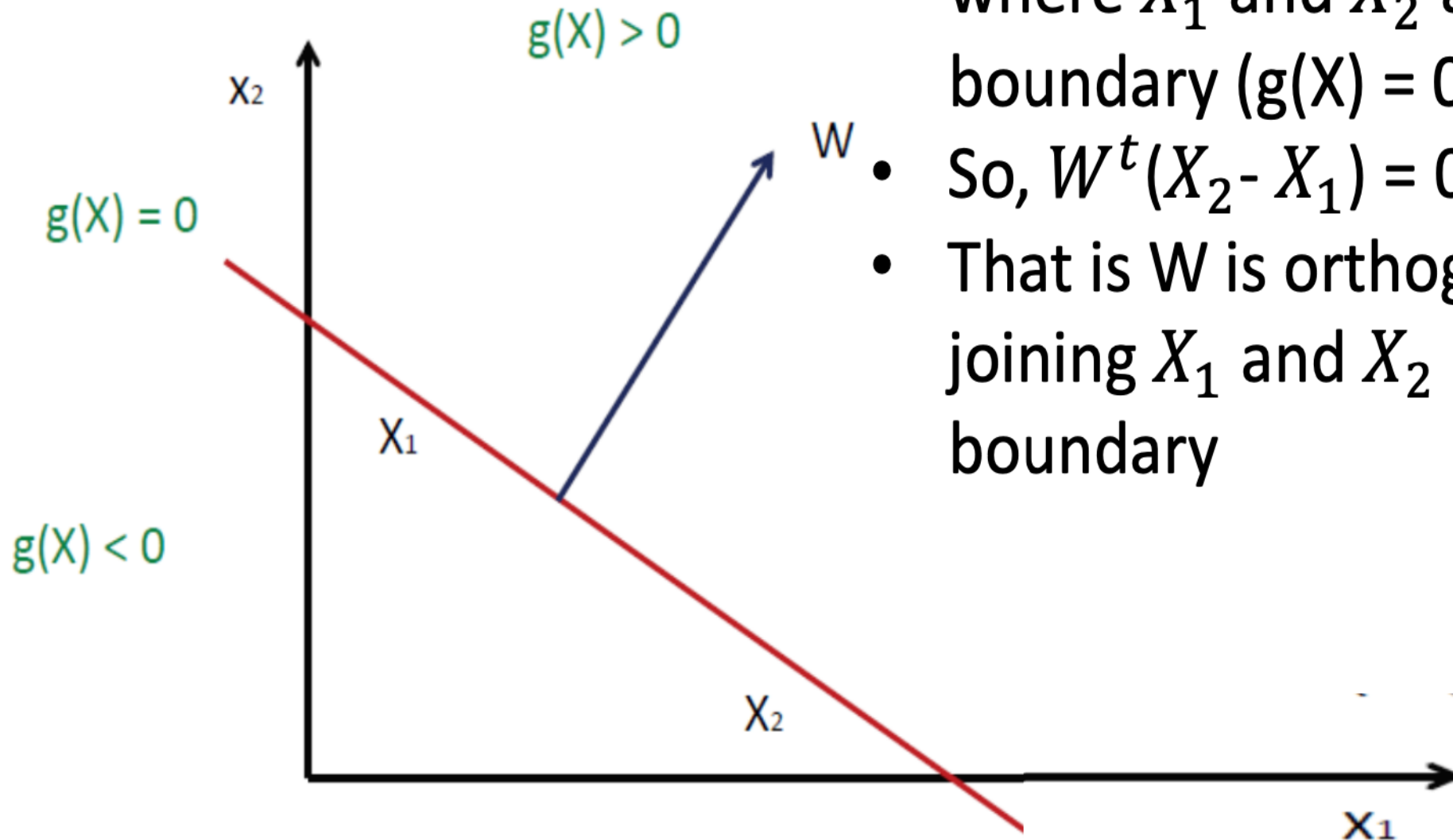


Augmented Vector

- Augmented: $(0,0) \rightarrow (1,0,0)$; $(0,1) \rightarrow (1,0,1)$; $(1,0) \rightarrow (1,1,0)$; $(1,1) \rightarrow (1,1,1)$
- We use X for X_a , for the sake of brevity, even when we use the augmented vector.
- $\{(X_i, y_i), i = 1, \dots, n\}$ is linearly separable if there is W_* such that $W_*^t X_i > 0$ if $y_i = 1$ else $y_i = 0$.
- Such a W_* characterizes a separating hyperplane; there could be infinitely many such W_* vectors.

Linear Classifier

- $g(X_1) = W^t X_1 + b = W^t X_2 + b = g(X_2)$ where X_1 and X_2 are on the decision boundary ($g(X) = 0$)
- So, $W^t (X_2 - X_1) = 0$
- That is W is orthogonal to the line joining X_1 and X_2 or the decision boundary



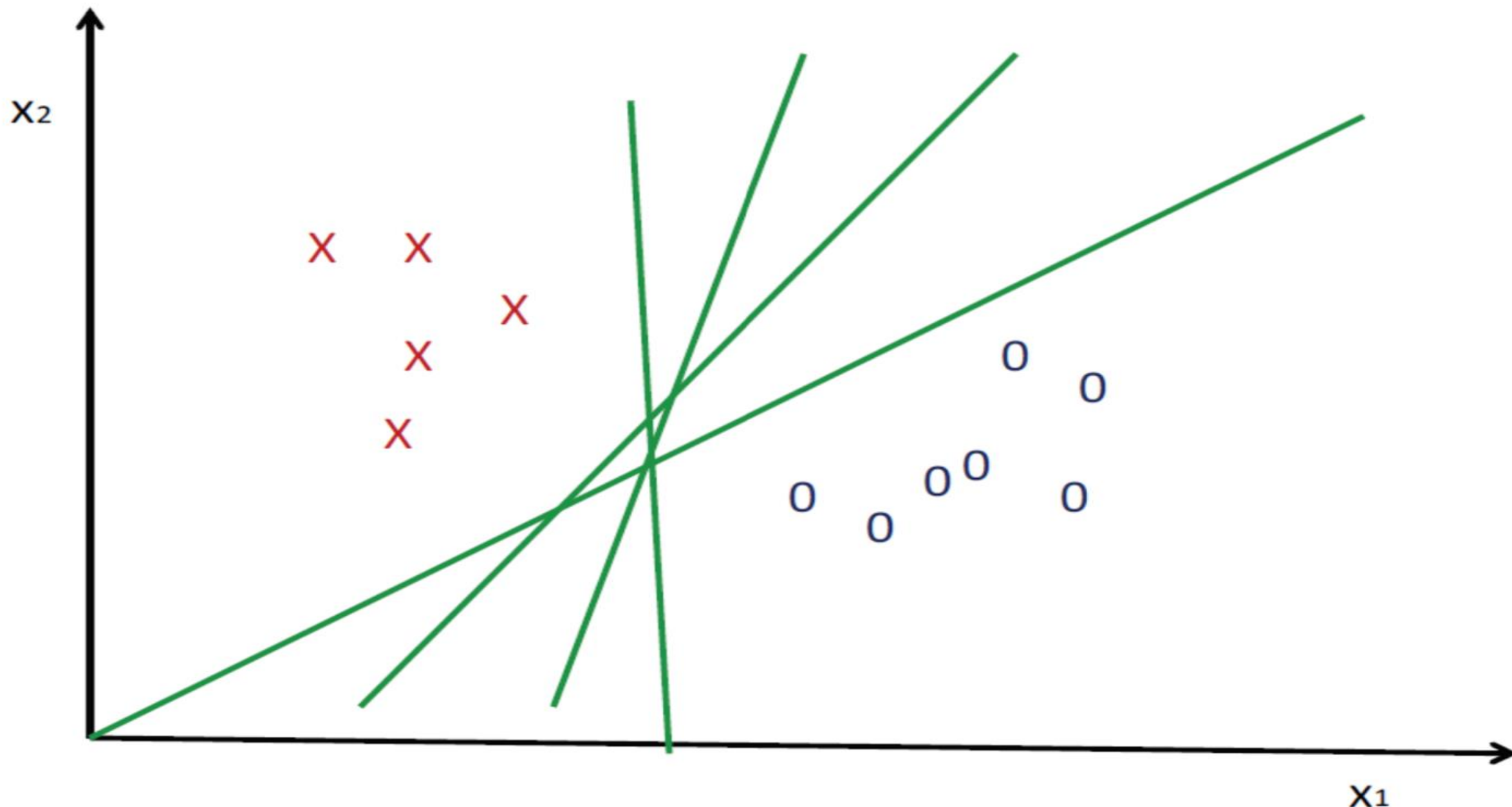
Linear Discriminant Functions

- Logical OR

x	y	In OR	$x + y > 0$	ExOR	$x + y - 2xy > 0$
0	0	0	0	0	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	1	1	0	0

- Let $\phi_i(X)$, $i = 1, \dots, m$ be some fixed functions
 - For example, $\phi_1(X) = x$; $\phi_2(X) = y$; $\phi_3(X) = xy$
 - $\text{ExclusiveOR}(x, y) = \phi_1(X) + \phi_2(X) - 2\phi_3(X)$
 - It is linear in W_i and $\phi_i(X)$
- So, $\sum_{i=1}^m W_i \phi_i(X) + W_0$ is also a linear discriminant function

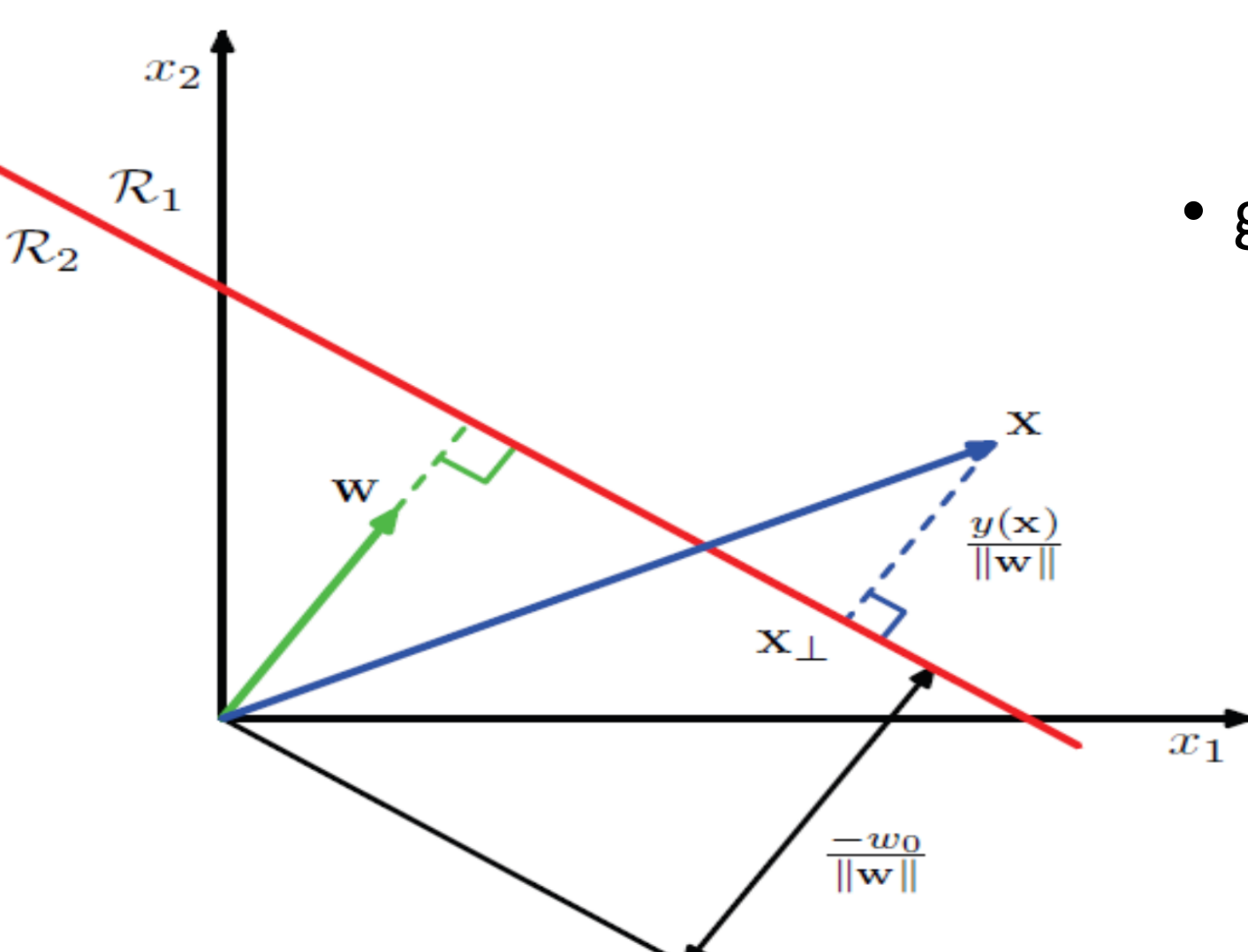
Linear Separators



Linear Separability

- Consider $\{((1,1),0), ((2,2),0), ((4,4),1)\}$; it is a linearly separable set.
- The augmented vectors are: $(1, 1, 1), (1, 2, 2), (1, 4, 4)$.
- A possible W is $(-6,1,1):-4,-2,2$. Another W is $(-12,2,3):-7,-2,8$
- $(-12, 2, 3)$ correctly classifies all the three patterns.
- However, if we add the pattern $((4, 1), 1)$, then both $(-6, 1, 1)$ and $(-12, 2, 3)$ fail to classify the new pattern correctly.
- Note that $(-12, 4, 1)$ classifies all the 4 patterns correctly.

Linear Classifier: Role of W and b



✦ Consider vector X . $X = X_{\perp} + r \frac{W}{\|W\|}$

- $g(X) = W^t X + b = W^t (X_{\perp} + r \frac{W}{\|W\|}) + b$

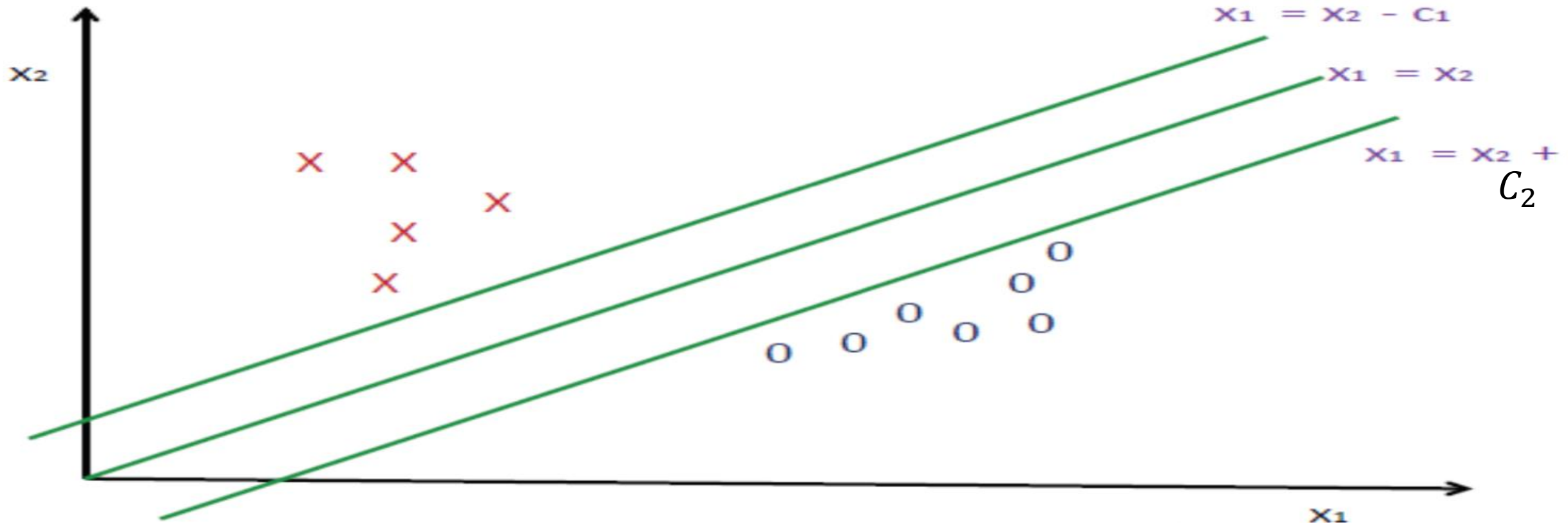
- So, $g(X) = W^t X_{\perp} + b + r \|W\|$

- $g(X) = 0 + r \|W\| \Rightarrow r = \frac{g(X)}{\|W\|}$

- The decision boundary goes through origin if $b=0$;
- $b > 0 \Rightarrow$ origin in +ve ($g(0) > 0$)
 $b < 0 \Rightarrow$ origin in negative half

- W decides **orientation** of the boundary as it is orthogonal to the boundary.
- b decides **location** of the decision boundary

Linear Separability



- Both classify all the patterns correctly.
- For $W = (C_1, 1, -1)$ origin is in the positive side and
- for $W = (-C_2, 1, -1)$ origin is in the negative half side.

Example of Linearly Separable Classes

Pattern No.		x_1	x_2	Class	$w^t x_a$
1	1	0.5	3.0	X	-3.5
2	1	1	3	X	-3.0
3	1	0.5	2.5	X	-3.0
4	1	1	2.5	X	-2.5
5	1	1.5	2.5	X	-2.0
6	1	4.5	1	O	2.5
7	1	5	1	O	3.0
8	1	4.5	0.5	O	3.0
9	1	5.5	0.5	O	4.0

- Consider the 9 patterns in the table: 5 from X and 4 from O
- Using $W = (-1, 1, -1)$ we get the values shown in the last column. Augmented data is used. So, all the augmented patterns are correctly classified.
- So, obtain W such that $W^t X_a > 0$ for positive class, else negative.

Linear Separability

- Consider the decision boundary given by $x_1 = x_2$. So, any point on this line is of the form (α, α) . So, the augmented vector X_a is $(1, \alpha, \alpha)$
- W here is $(0, 1, -1)$ based on the equation for the decision boundary, which is $x_1 - x_2 = 0$
- Note that $W^t X_a = 0$. So, W is orthogonal to the dec. boundary $(0 * 1 + 1 * \alpha - 1 * \alpha = 0)$
- The cosine of the angle between W and any positive pattern is positive.
- For example, for pattern number 7, the augmented vector O_a is $(1, 5, 1)$ and the cosine of the angle is $\frac{4}{\sqrt{54}} \left(\frac{W^t O_a}{||W|| ||O_a||} \right)$, which is positive. So, positive points are located at an angle between -90 and 90 from W

Normalization

- W for the separating line is such that $W^t X_a < 0$ for negative class patterns and $W^t O_a > 0$ for positive class patterns.
- A simple way to deal with all the patterns uniformly is to normalize the patterns.
- All the positive class patterns are already normalized, O_a is not affected. Each negative class pattern X_a is normalized by replacing it by $-X_a$.
- For a negative pattern X_a , $W^t X_a < 0$; so, $W^t (-X_a) > 0$. So, after normalization, all the patterns will be on the positive side of the decision boundary.

Augmented and Normalized Data

Pattern	1	x_1	x_2	$W^t X_a$
1	-1	-0.5	-3.0	2.5
2	-1	-1	-3	2
3	-1	-0.5	-2.5	2.0
4	-1	-1	-2.5	1.5
5	-1	-1.5	-2.5	1.0
6	1	4.5	1	3.5
7	1	5	1	4.0
8	1	4.5	0.5	4.0
9	1	5.5	0.5	5.0

- All the 9 patterns in the Table are augmented and normalized.
- Using $W = (0, 1, -1)$, we get the values returned by the linear discriminant function as shown in the last column.

Augmented and Normalized Data

- Let X_a^n be the augmented and normalized version of X belonging to the *negative class*.
- Let O_a^n be the augmented and normalized version of O belonging to the *positive class*.
- If W_{opt} is the weight vector that classifies all the patterns correctly, then $W_{opt}^t X_a^n > 0$ and $W_{opt}^t O_a^n > 0$ for all the patterns, whether from negative or positive class.
- So we use augmented and normalized patterns and learn W that correctly classifies all the patterns. For the sake of brevity we consider all the patterns as X 's henceforth.

Classification Using a Linear Discriminant

- Classification is based on $\text{sign}(W^t X)$ where X is augmented but not normalized.
- Need to learn the Optimal W for the training patterns.
- Perceptron Algorithm is one of the earliest algorithms for learning a linear discriminant function.
- Find a W vector and the corresponding hyperplane.

Perceptron¹

(MIT AI LAB started in late 50s)



Cornell Aeronautical
Laboratory

Frank Rosenblatt (1928 – 1971)

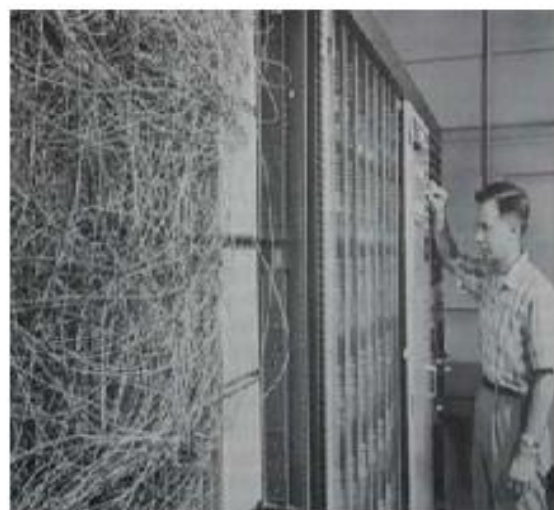
[1] F. Rosenblatt, The perceptron: A probabilistic model for information storage and organization in the brain, *Psychological Review*, Nov 1958, 386-408.

Perceptron¹

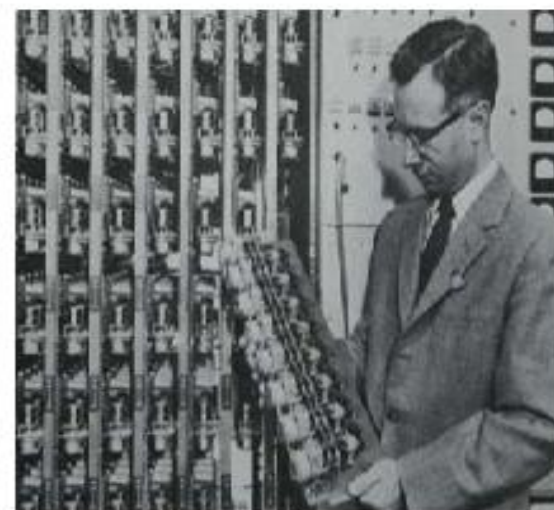
- Classification is based on $\text{sgn}(\sum_{i=1}^d w_i x_i + w_0) = \text{sgn}(W^t X)$
- Find weighted sum and threshold, that is $W^t X > 0$



Visual Inputs



Patch board allowing
configuration of inputs ϕ



Rack of adaptive weights w
(motor-driven potentiometers)

[1] F. Rosenblatt, The perceptron: A probabilistic model for information storage and organization in the brain, Psychological Review, Nov 1958, 386-408.

Perceptron Learning Algorithm¹

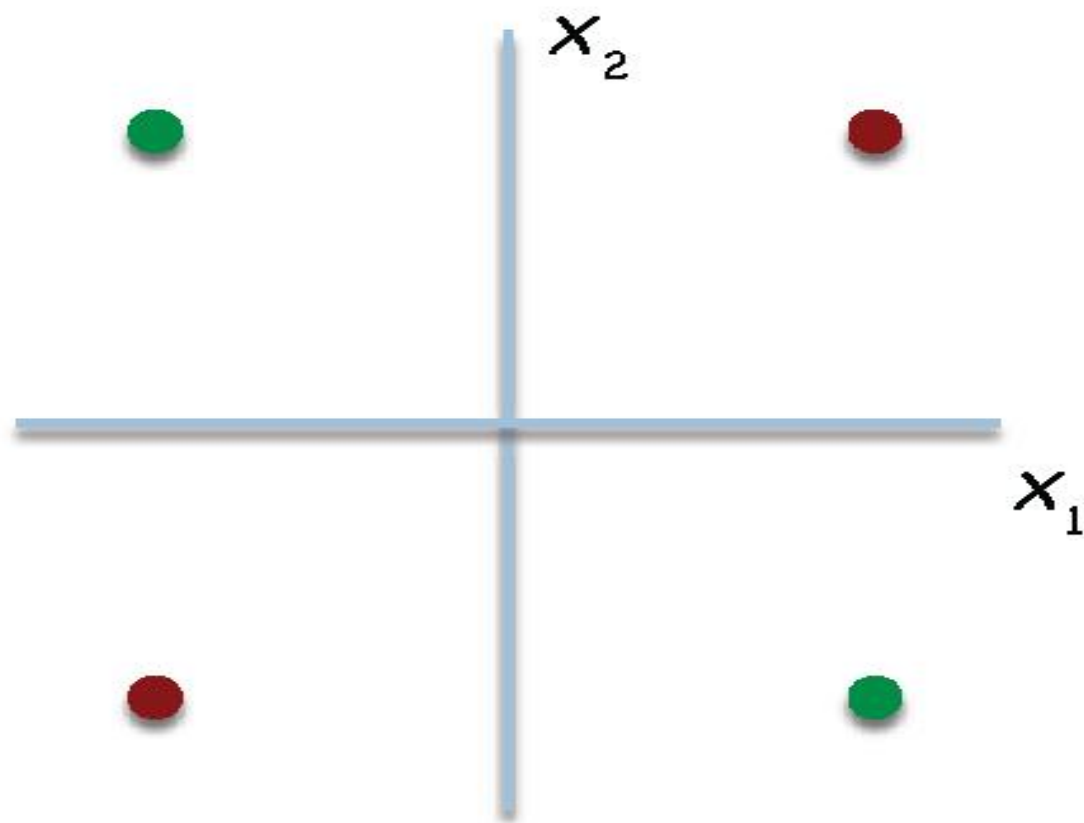
Learning is achieved through change of weights

- ❶ Initialize W as the null vector; set $i = 0$ and $W_i = \mathbf{0}$.
- ❷ For $j = 1$ to n do
If $W_i^t X_j \leq 0$ then $i = i + 1$ and $W_i = W_{i-1} + X_j$
- ❸ Repeat step 2 till the value of i does not change over an entire iteration (epoch) over all the n patterns.

This algorithm converges to a required W in a finite number of epochs if the classes are linearly separable.

[1] M. N. Murty and V. S. Devi, Pattern Recognition: An algorithmic Approach, Springer, 2011

Perceptrons?¹



Marvin Minsky

[1] M. L. Minsky and S. papert: Perceptrons: An Introduction to Computational Geometry, MIT Press, 1988.

Order of Perceptron¹

- Boolean functions like **exclusive or** can not be realized using functions that are linear in the variables .
- Any boolean function can be represented as a positive normal form, a **sum of minterms**.
 - $XOR(X_1, X_2) \Leftrightarrow X_1\bar{X}_2 + \bar{X}_1X_2 \Leftrightarrow X_1 + X_2 - 2X_1X_2 > 0$. So, **XOR** is linear in X_1 , X_2 , and X_1X_2 but not in X_1 and X_2 alone. The corresponding perceptron is of order 2.
 - $OddParity(X_1, X_2, X_3) \Leftrightarrow X_1 + X_2 + X_3 - 2X_1X_2 - 2X_2X_3 - 2X_1X_3 + 4X_1X_2X_3 > 0$. The order of this perceptron is 3. Further all the minterms of size less than or equal to 3 are used in the representation.
 - So, any boolean function over d variables is represented using 2^d minterms.
- Some functions are simpler. $X_1 \vee X_2 \Leftrightarrow X_1 + X_2 > 0$ and $X_1 \wedge X_2 \Leftrightarrow X_1 + X_2 > 1$.

[1] M. L. Minsky and S. papert: Perceptrons: An Introduction to Computational Geometry, MIT Press, 1988.

Linearity is Adequate

- Support Vector Machines have popularized the notion of linear classifiers by employing KERNEL TRICK.

x	y	xor(x,y)	x + y - 2xy
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

- Non-linear in 2-D:** $X = \begin{pmatrix} x \\ y \end{pmatrix}$ $\text{Xor}(x,y) = y\bar{x} + x\bar{y}$
- Linear in 3-D:** $X = \begin{pmatrix} x \\ y \\ xy \end{pmatrix}$ $\text{xor}(X) = x + y - 2xy$

Non-Linear to Linear?

- $f(x) = a_0 + a_1x + \dots + a_nx^n$: non-linear in x

- $W = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix}$ and $X = \begin{pmatrix} 1 \\ x \\ \vdots \\ x^n \end{pmatrix}$

- $f(x) = W^t X$: linear in X ; that is in a higher-dimensional space
- Both SVMs and NBC are linear classifiers
- SVM has become the most popular classifier exploiting the notion of Dot Product (Inner Product on Reals).
- Linearity is a matter of convenience and even when the data is not linearly separable, non-linear SVM is avoided to simplify the computation.

Perceptron Learning: An Example

x	y	x ∨ y
0	0	0
0	1	1
1	0	1
1	1	1

Truth Table of OR

1	x	y	x ∨ y
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Augmented

1	x	y	x ∨ y
-1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Augmented and Normalized

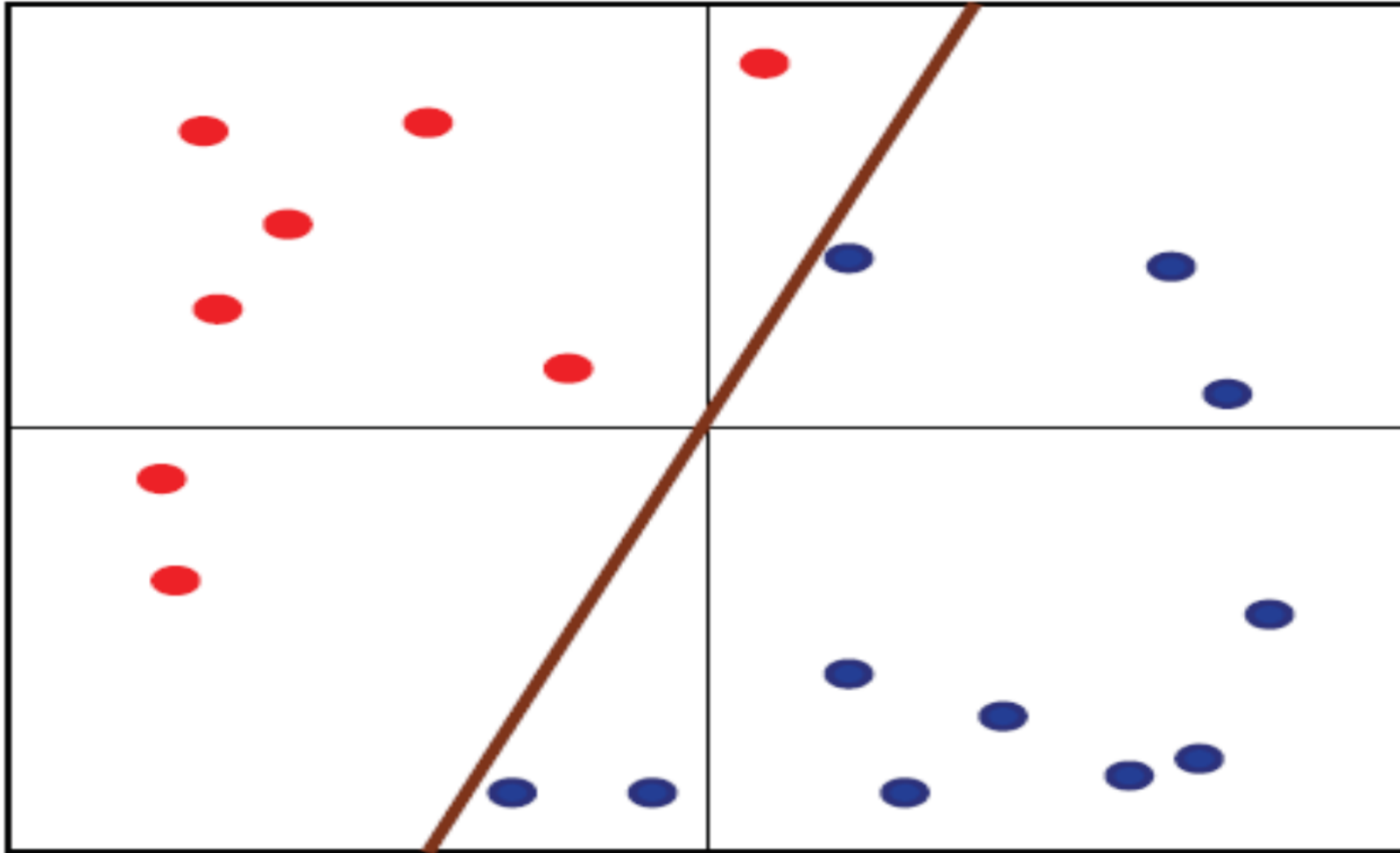
Weight Vector W	First Pattern Misclassified (X)	$W^t X$	Weight Vector W	First Pattern Misclassified (X)	$W^t X$
$W_0 = (0, 0, 0)$	$(-1, 0, 0)$	0	$W_5 = (-1, 1, 1)$	$(1, 0, 1)$	0
$W_1 = (-1, 0, 0)$	$(1, 0, 1)$	-1	$W_6 = (0, 1, 2)$	$(-1, 0, 0)$	0
$W_2 = (0, 0, 1)$	$(1, 1, 0)$	0	$W_7 = (-1, 1, 2)$	$(1, 1, 0)$	0
$W_3 = (1, 1, 1)$	$(-1, 0, 0)$	-1	$W_8 = (0, 2, 2)$	$(-1, 0, 0)$	0
$W_4 = (0, 1, 1)$	$(-1, 0, 0)$	0	$W_9 = (-1, 2, 2)$	None	$>0 \forall X$

Perceptron Learning: Intuition Behind

- Let W_i be the weight vector that misclassified a pattern X_j . That is $W_i^t X_j \leq 0$
- So $W_{i+1} = W_i + X_j$. This means $W_{i+1}^t X_j = W_i^t X_j + X_j^t X_j$
- As a consequence $W_{i+1}^t X_j$ is larger than $W_i^t X_j$ by $\|X_j\|^2$ because $\|X_j\|^2$ is positive.
- So, W_{i+1} is better suited to classify X_j than W_i and $W_{i+1}^t X_j$ can be positive even if $W_i^t X_j$ is not.

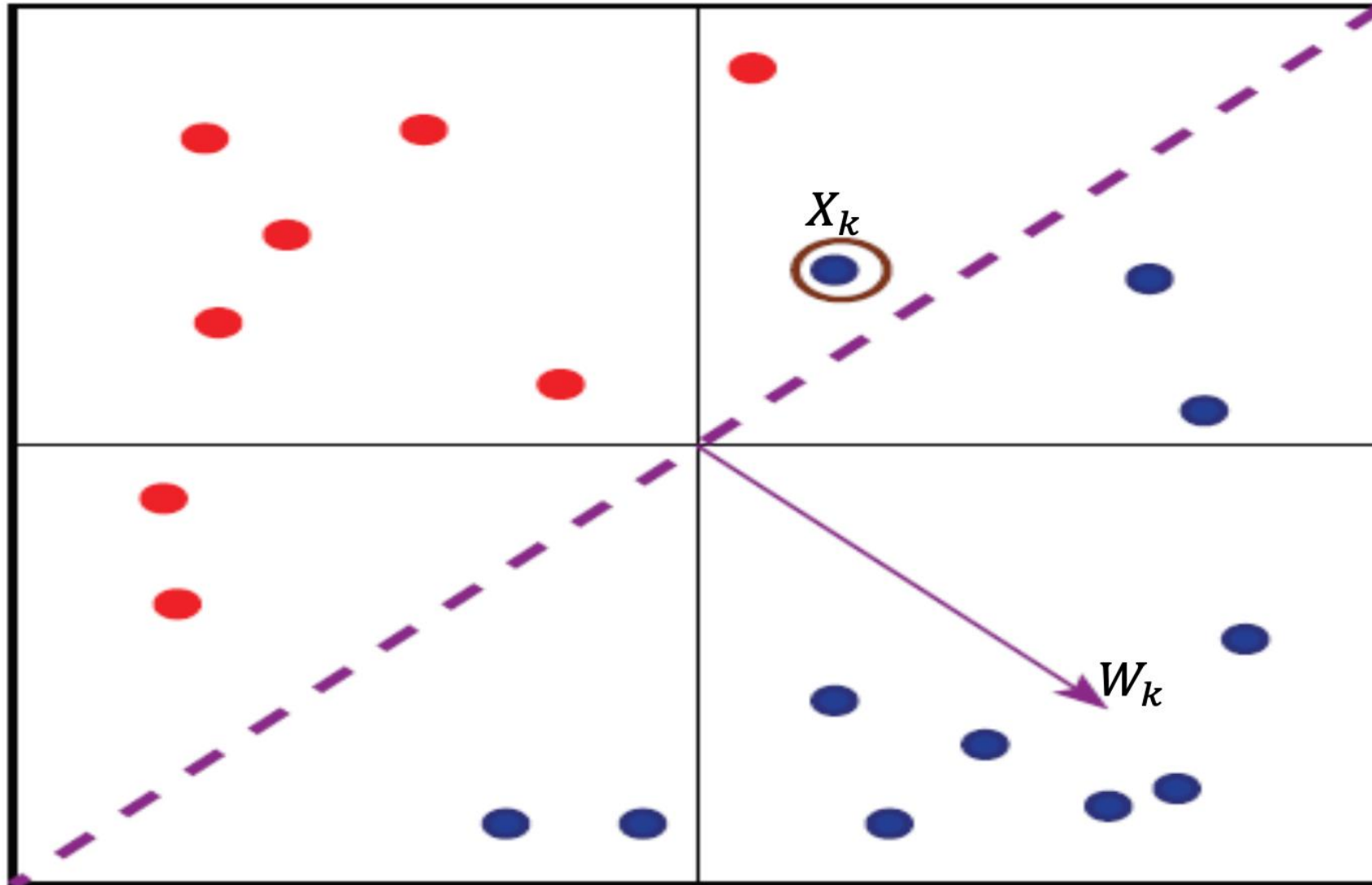
Perceptron Learning: A Geometric View

The algorithm has a simple geometric view. Consider the following dataset.



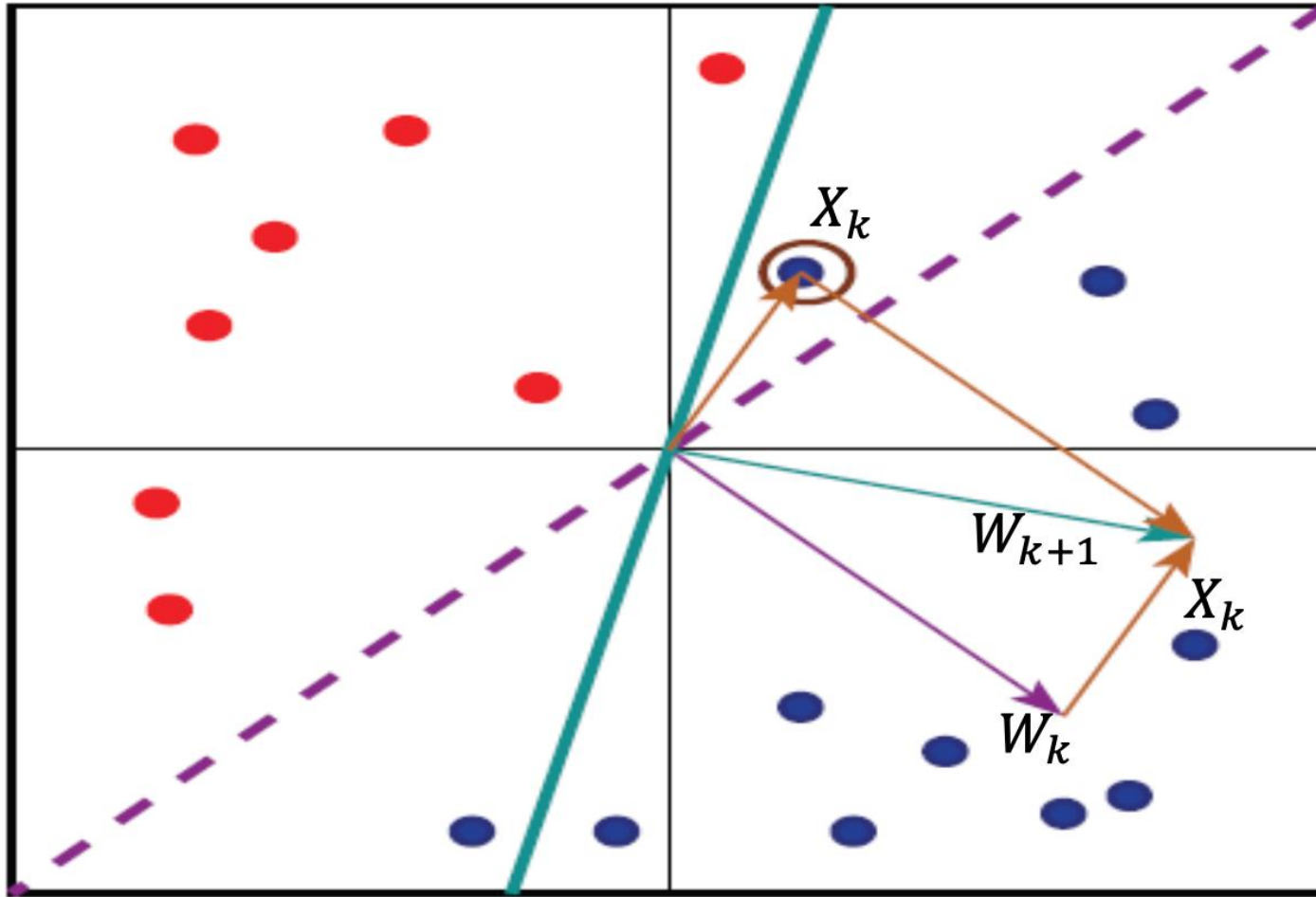
Perceptron Learning: A Geometric View

Suppose W_k misclassifies a pattern



Perceptron Learning: A Geometric View

- Now the correction made to W_k may be visualized as



- Note that X_k is correctly classified by W_{k+1}

Perceptron Learning: Non-Linearly Separable Classes

Pattern No	Value of x	Class	Pattern No	1	x	x ²	Class
1	1	O	1	1	1	1	O
2	-1	O	2	1	-1	1	O
3	2	O	3	1	2	4	O
4	-2	O	4	1	-2	4	O
5	3	X	5	-1	-3	-9	X
6	4	X	6	-1	-4	-16	X
7	-3	X	7	-1	3	-9	X
8	-4	X	8	-1	4	-16	X

We are given the form of the function $a + bx + cx^2$. The discriminant function is of the form $W^t X$ where $W = (a, b, c)$ and $X = (1, x, x^2)$. We get optimal $W = (17, 0, -2)$ using the perceptron algorithm, which classifies all the 8 patterns correctly. Note that the final form is if $17 - 2x^2 > 0$ then the class is O else it is X.

Multi-Class Problems

- Linear discriminants are ideally suited for two-class or binary classification problems.
- It is possible to extend a binary classifier to deal with C -class ($C > 2$) problems.
- Consider a pair of classes at a time. There are $\frac{C(C-1)}{2}$ pairs. Learn a linear discriminant for each pair of classes. Combine the decisions of all these classifiers to arrive at the class label.
- For each class C_i , let $\bar{C}_i = \bigcup_{j=1; j \neq i}^C C_j$ (the collection of patterns in all other classes) be the complementary class. Learn a **linear discriminant function** to classify **between C_i and $\bar{C}_i$** for each i . **Combine these linear discriminants** to classify.

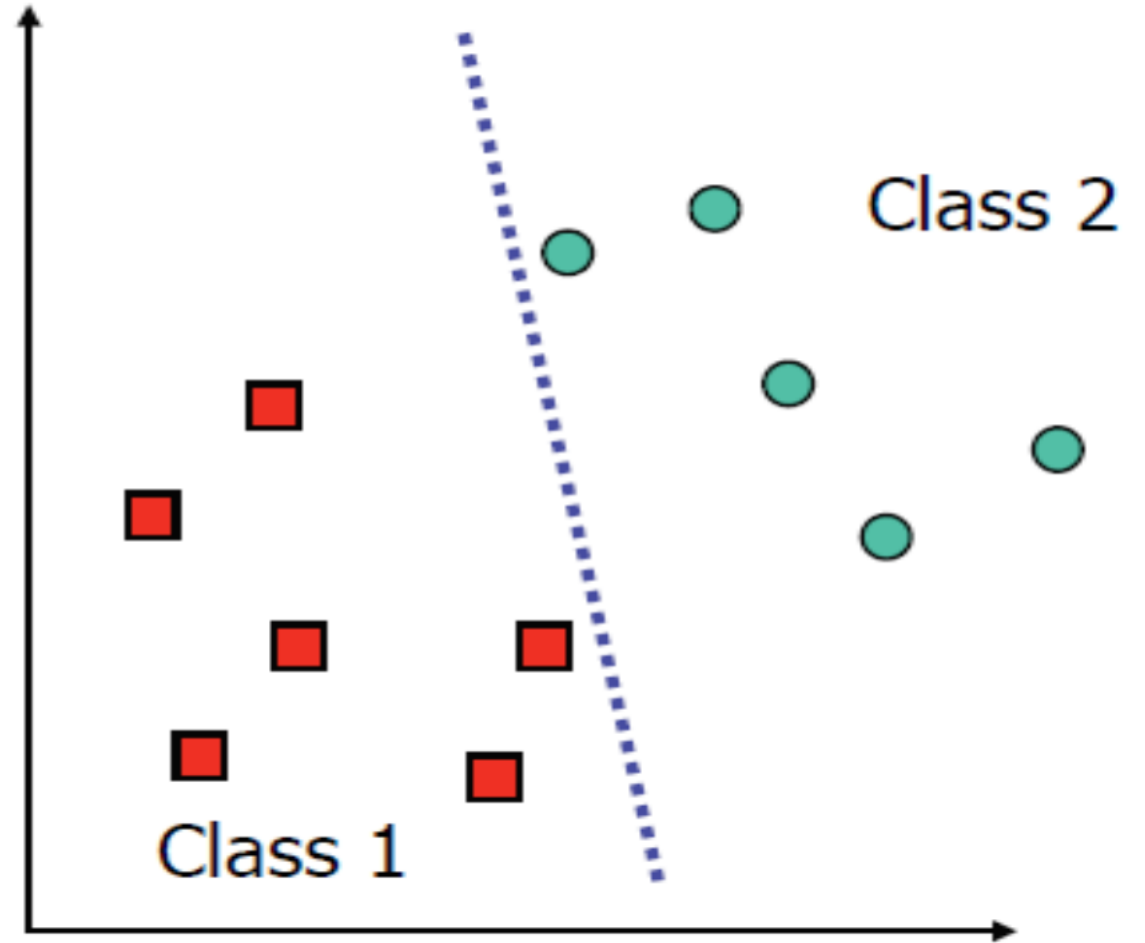
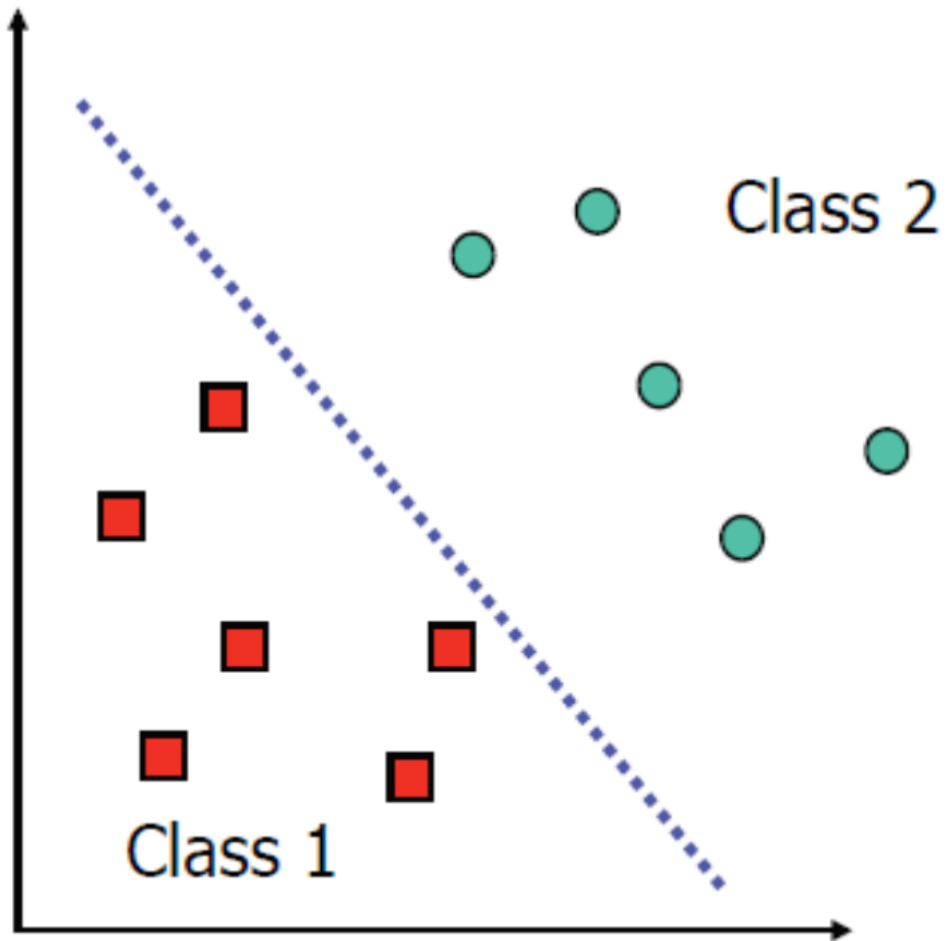
SVM Inventor

" IF YOU POSSESS A **RESTRICTED**
AMOUNT OF INFORMATION FOR SOLVING
SOME PROBLEM, **TRY TO SOLVE THE**
PROBLEM DIRECTLY AND NEVER SOLVE A
MORE GENERAL ONE AS AN INTERMEDIATE
STEP. IT IS POSSIBLE THAT **THE**
AVAILABLE INFORMATION IS SUFFICIENT
FOR A DIRECT SOLUTION BUT IS
INSUFFICIENT FOR SOLVING A MORE
GENERAL INTERMEDIATE PROBLEM."

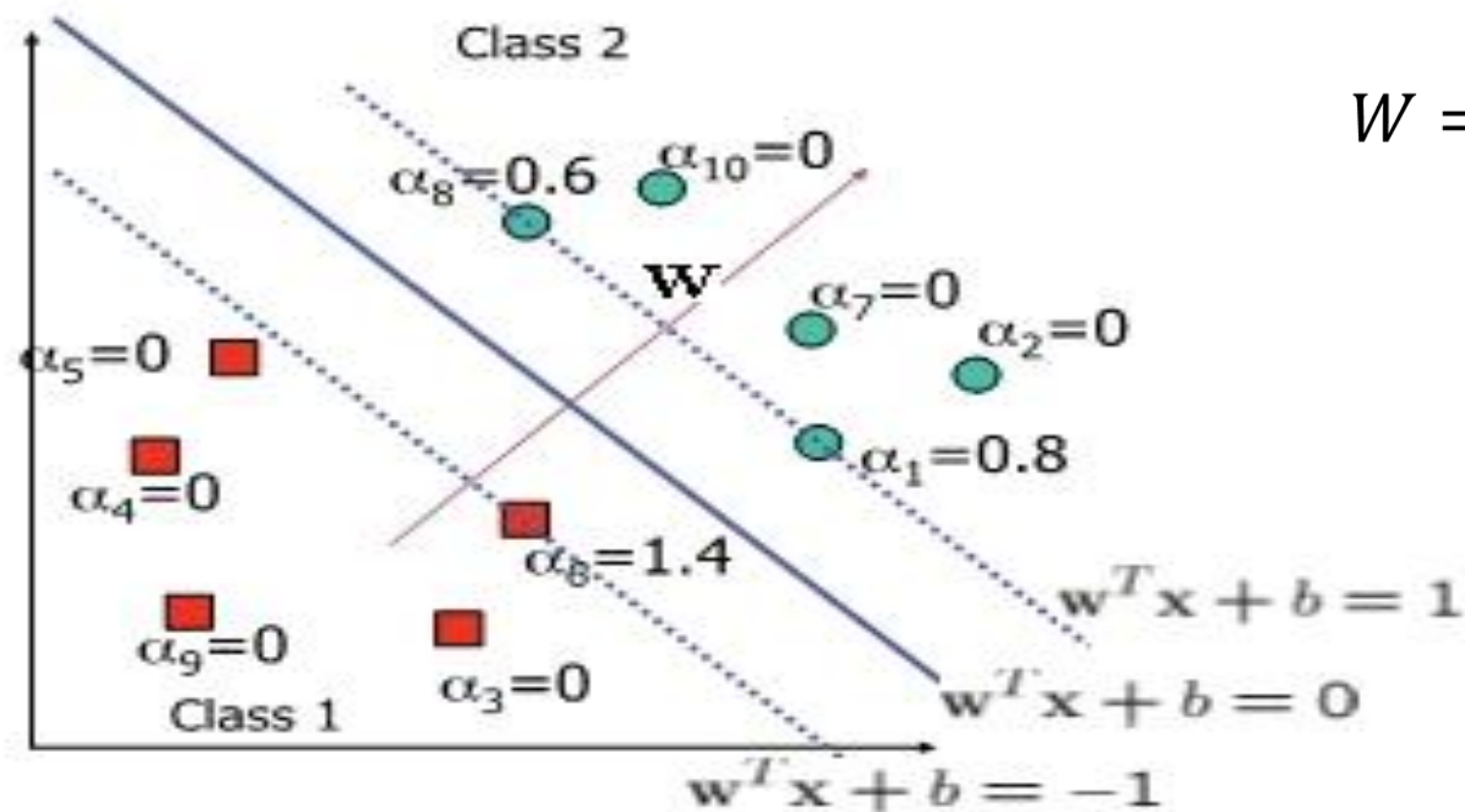
SVM inventor, Vladimir Vapnik



Bad Decision Boundaries



Geometric Interpretation¹



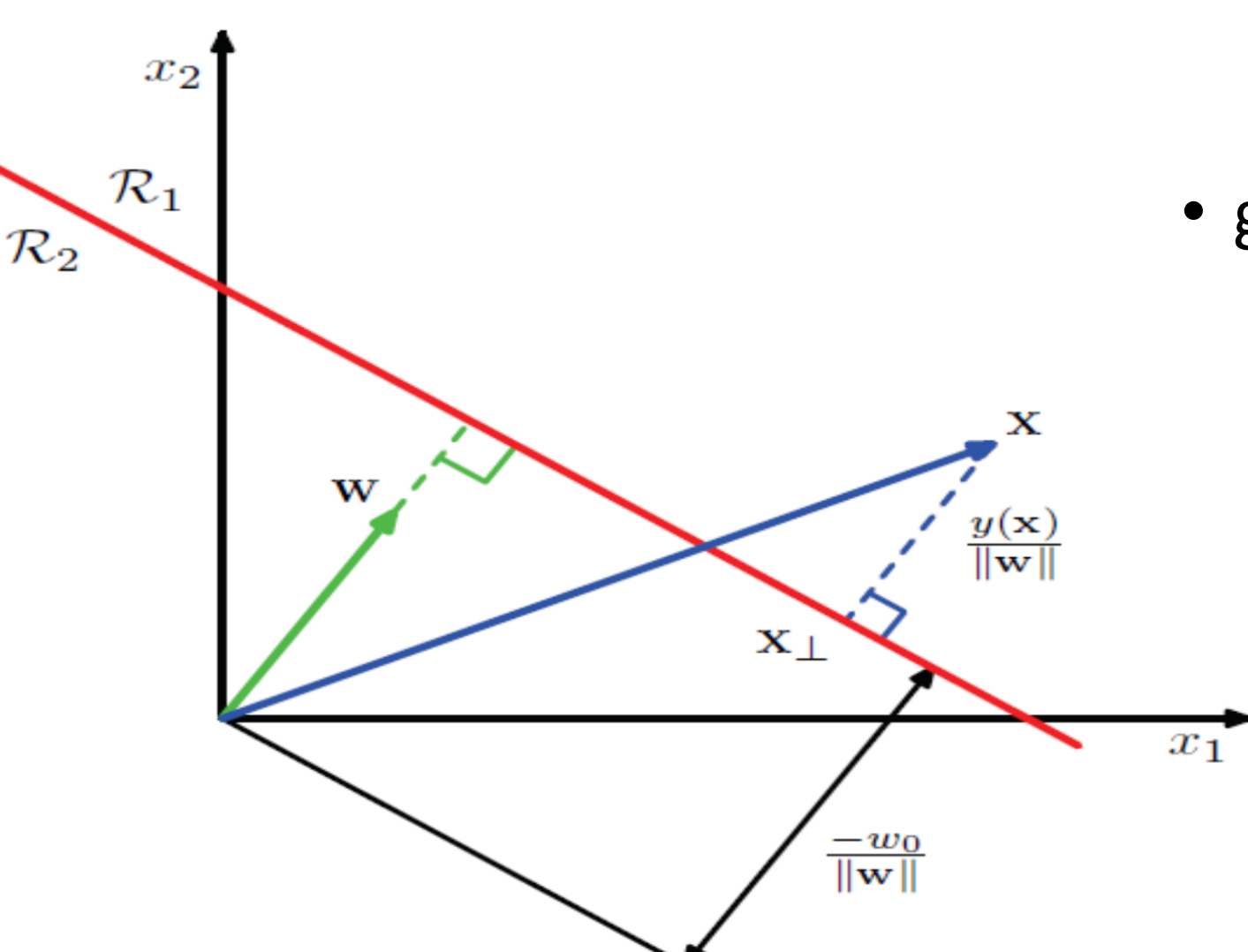
$$W = \sum_i \alpha_i x_i y_i$$

- Note that $S = \{X_1, X_6, X_8\}$

Support Vector Machines (SVMs)

- Learning an optimal W and the corresponding decision boundary.
- Maximizing the separation between classes is based on a well-behaved optimization problem. It is called Maximum Margin problem.
- SVM Can map input vectors into a high dimensional feature space. Hence can learn nonlinear decision boundaries in the input space.
- It employs kernel trick by using a suitable kernel (similarity) function.
- SVM was first introduced in 1992. SVM is now regarded as an important example of kernel methods, one of the key areas in ML.
- SVM became popular because of its success in handwritten digit recognition; 1.1% test error rate for SVM. This is the same as the error rates of a carefully constructed neural network.

Linear Classifier: Role of W and b

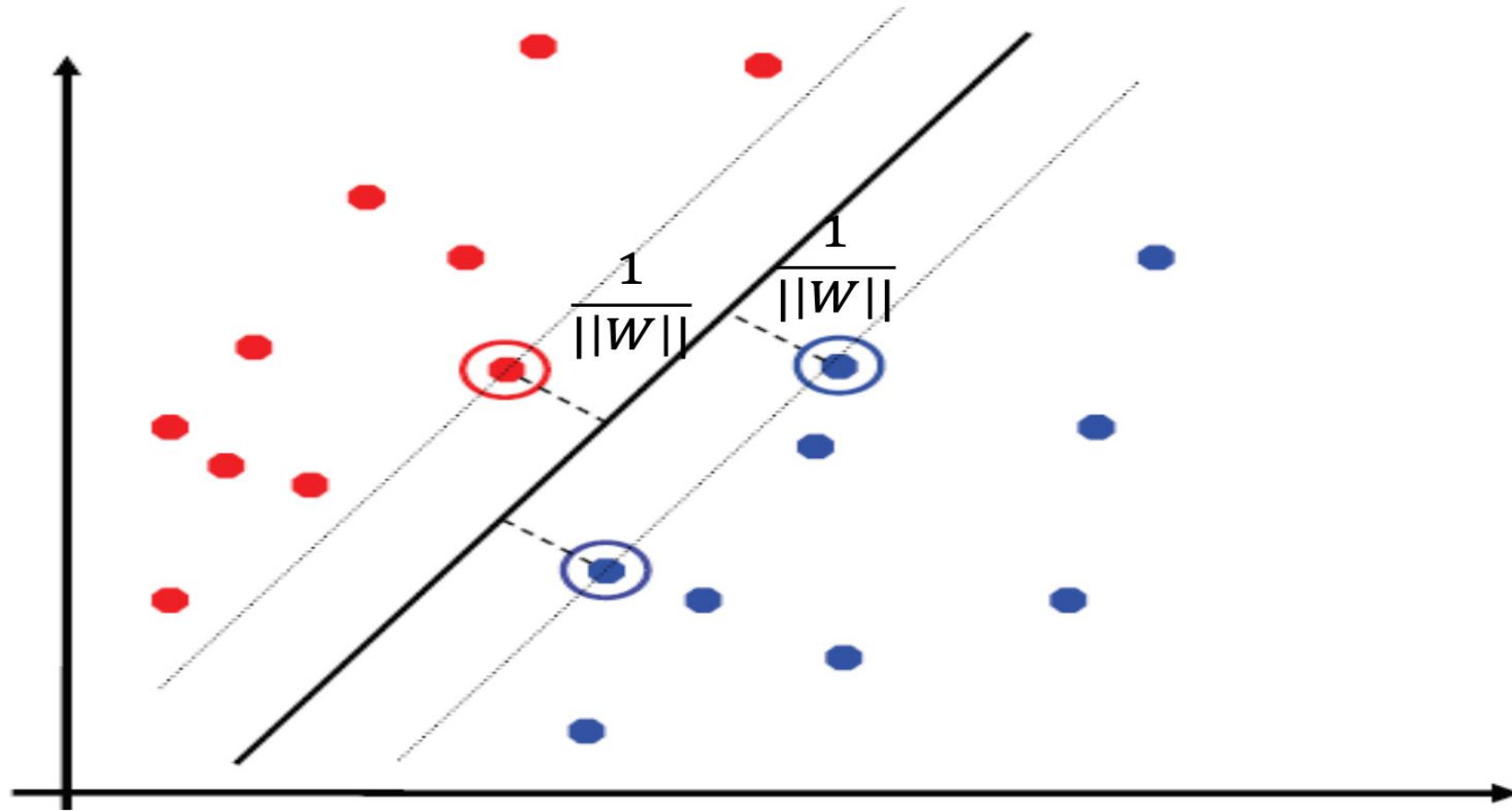


✦ Consider vector X . $X = X_{\perp} + r \frac{W}{\|W\|}$

- $g(X) = W^t X + b = W^t (X_{\perp} + r \frac{W}{\|W\|}) + b$
 - So, $g(X) = W^t X_{\perp} + b + r \|W\|$
 - $g(X) = 0 + r \|W\| \Rightarrow r = \frac{g(X)}{\|W\|}$
- The decision boundary goes through origin if $b=0$;
 $b > 0 \Rightarrow$ origin in positive
 $b < 0 \Rightarrow$ origin in negative

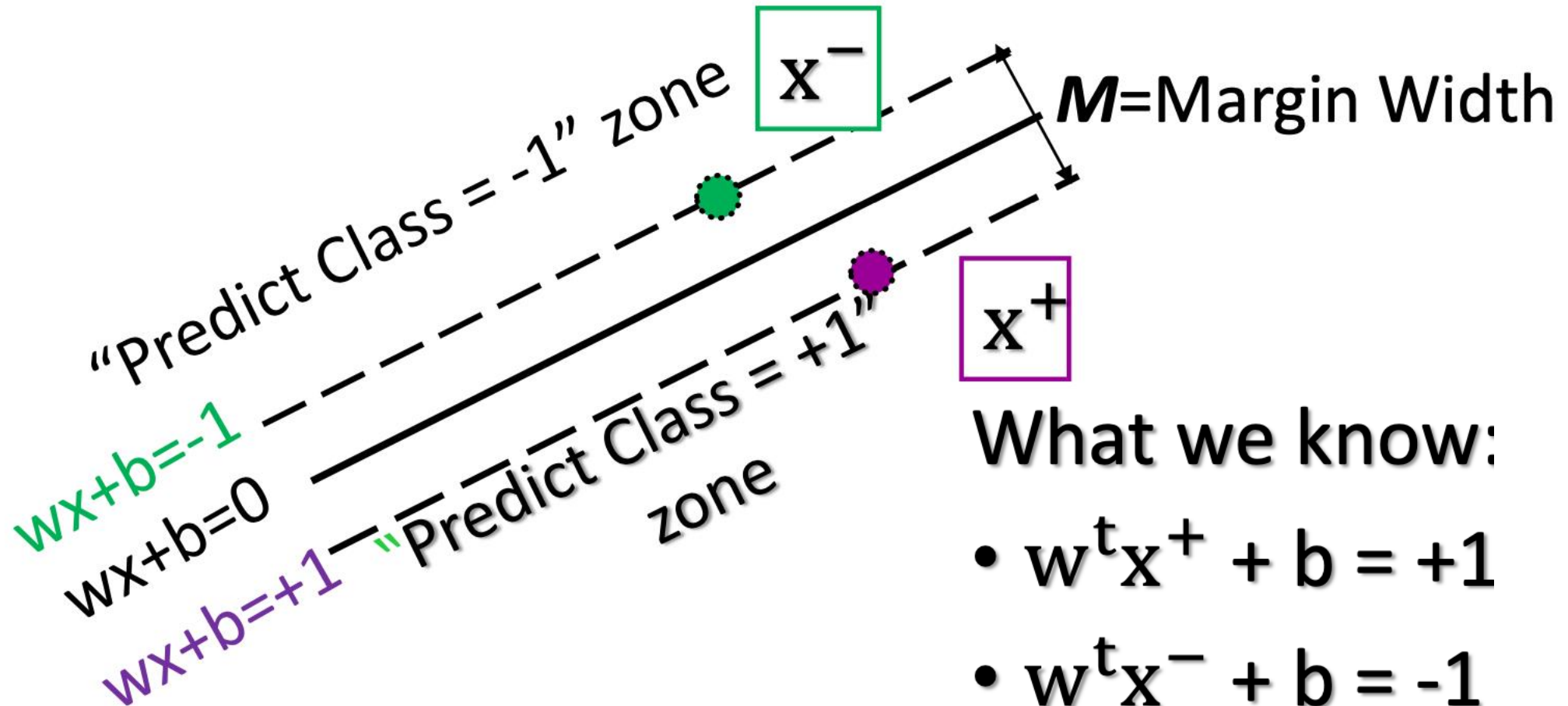
- W decides **orientation** of the boundary as it is orthogonal to the boundary.
- b decides **location** of the decision boundary

Large Margin Decision Boundary



- Distance from X to the separator is $\frac{W^t X + b}{\|W\|}$.
- Examples closest to the hyperplane are **support vectors**.
- Margin of the separator is the distance between support vectors of different classes.

SVM Margin



$$\text{Margin} = M = \frac{(x^+ - x^-) \cdot w}{|w|} = \frac{2}{|w|}$$

What we know:

- $w^t x^+ + b = +1$
- $w^t x^- + b = -1$
- $w^t (x^+ - x^-) = 2$

Linearly Separable Case

- Because of linear separability, we have

$$W^t X_i + b > 0, \forall i \text{ s.t. } y_i = +1$$

$$W^t X_i + b < 0, \forall i \text{ s.t. } y_i = -1$$

- The training set is finite, $\exists \epsilon > 0$ s.t.

$$W^t X_i + b \geq \epsilon, \forall i \text{ s.t. } y_i = +1$$

$$W^t X_i + b \leq -\epsilon, \forall i \text{ s.t. } y_i = -1$$

- So, we can scale W and b to realize

$$W^t X_i + b \geq +1, \forall i \text{ s.t. } y_i = +1$$

$$W^t X_i + b \leq -1, \forall i \text{ s.t. } y_i = -1$$

- Equivalently, we have

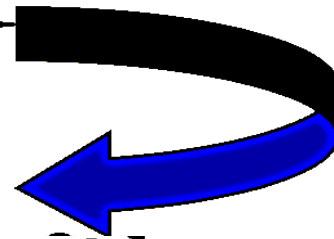
$$y_i(W^t X_i + b) \geq 1, \forall i \text{ (note that } y_i \in \{+1, -1\})$$

Linearly Separable Case

- For a linearly separable training set, any decision boundary characterized by W and b can be suitably scaled so that $y_i(W^t X_i + b) \geq 1, \forall i$
- In the linearly separable case, there are no training patterns between the two parallel hyperplanes characterized by $W^t X + b = +1$ and $W^t X + b = -1$
- Distance between these two hyperplanes, called the margin, is given by $\frac{2}{\|W\|}$.

Maximizing the Margin

- Goal: **1) Correctly classify all training data**

$$\left. \begin{aligned} wx_i + b &\geq 1 & \text{if } y_i = +1 \\ wx_i + b &\leq -1 & \text{if } y_i = -1 \end{aligned} \right\}$$

$$y_i(wx_i + b) \geq 1 \quad \text{for all } i$$

2) Maximize the Margin $M = \frac{2}{|w|}$

same as minimize $\frac{1}{2} w^t w$

We can formulate a Quadratic Optimization Problem and solve for W and b

$$\text{Minimize} \quad \Phi(w) = \frac{1}{2} w^t w$$

$$\text{subject to} \quad y_i(wx_i + b) \geq 1 \quad \forall i$$

SVM Optimization Problem

A separating hyperplane with the largest margin is the optimal hyperplane. So, the optimal hyperplane or the decision boundary corresponds to the solution of the following optimization problem with quadratic cost function and linear inequality constraints

Find $W \in \mathbb{R}^m$, $b \in \mathbb{R}$ to

$$\text{minimize } \frac{1}{2} W^t W$$

$$\text{subject to } y_i(W^t X_i + b) \geq 1, \quad i = 1, \dots, n$$

Note that $W^t W = ||W||^2$

SVM Optimization Problem

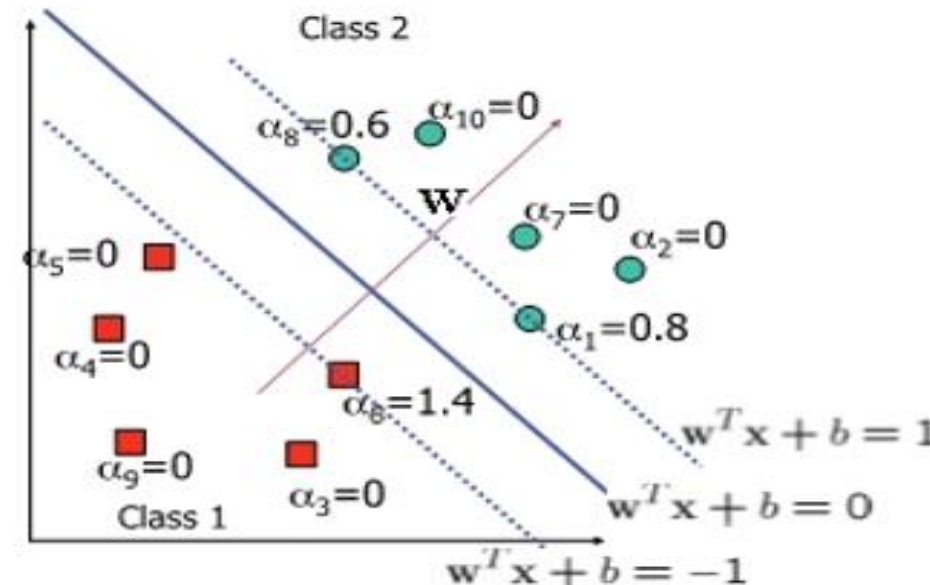
- minimize $\frac{1}{2} ||W||^2$
subject to $1 - y_i(W^t X_i + b) \leq 0$ for $i = 1, \dots, n$
- The Lagrangian is
$$\mathcal{L} = \frac{1}{2} W^t W + \sum_{i=1}^n \alpha_i (1 - y_i(W^t X_i + b)).$$
- The Kuhn-Tucker conditions give us
(by setting the gradient of $\mathcal{L}$ w.r.t. W to zero, we get)

$$W + \sum_{i=1}^n \alpha_i (-y_i) X_i = 0 \Rightarrow W = \sum_{i=1}^n \alpha_i y_i X_i$$

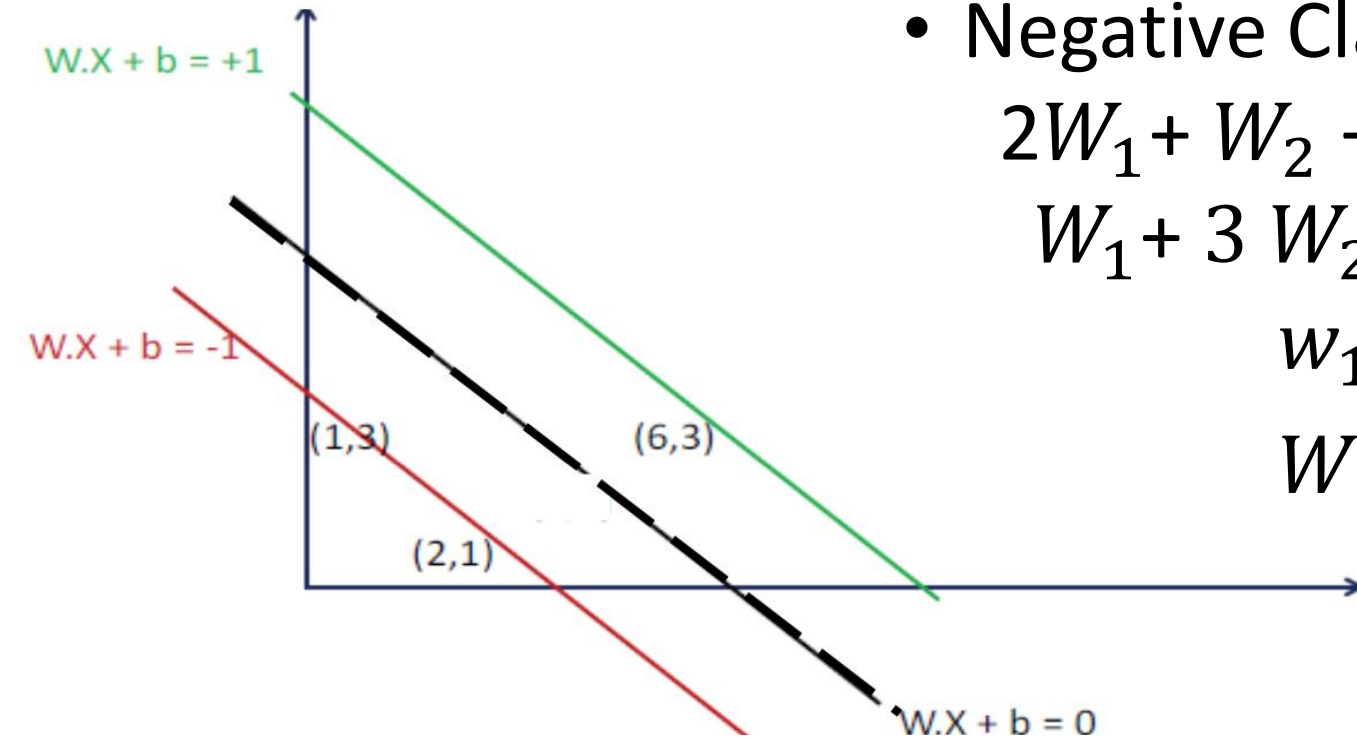
$$\frac{\delta \mathcal{L}}{\delta b} = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

$$1 - y_i(W^t X_i + b) \leq 0, \forall i$$

$$\alpha_i \geq 0 \text{ and } \alpha_i (1 - y_i(W^t X_i + b)) = 0, \forall i$$



SVM Optimization Problem: Example



• Negative Class:

$$2W_1 + W_2 + b = -1$$

$$W_1 + 3W_2 + b = -1$$

• Positive Class:

$$6W_1 + 3W_2 + b = +1$$

$$w_1 - 2w_2 = 0; 5w_2 + b = -1; 15w_2 + b = 1$$

$$W^t = \left(\frac{2}{5}, \frac{1}{5}\right) \text{ and } b = -2$$

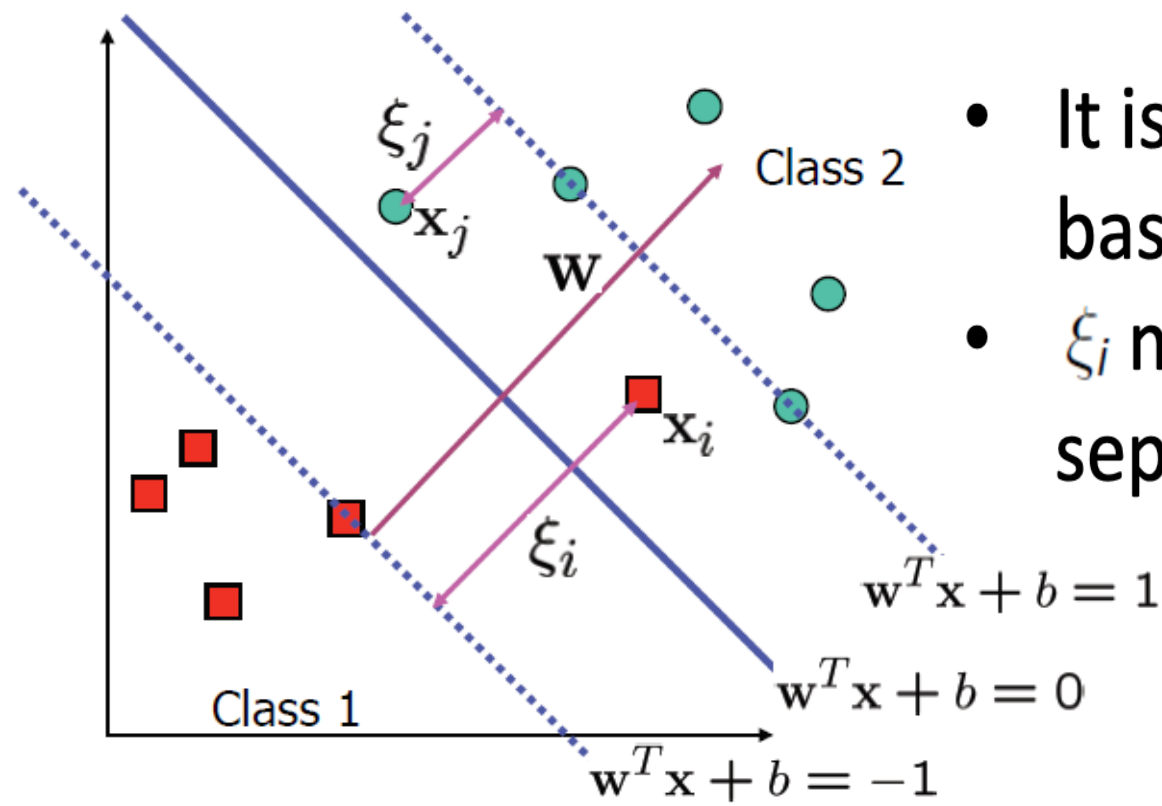
- $(4, 2)^t$ is on the boundary and note that $W^t(4, 2)^t + b = \left(\frac{2}{5}, \frac{1}{5}\right)(4, 2)^t - 2 = 0$. Similarly $(7, 1)^t$ is in the positive class and $(2, 0)^t$ is in the negative class.
- Further, $\sum_i \alpha_i y_i = 0 \Rightarrow -\alpha_1 - \alpha_2 + \alpha_3 \Rightarrow \alpha_3 = \alpha_1 + \alpha_2$
- $W^t = \left(\frac{2}{5}, \frac{1}{5}\right) = -\alpha_1(1, 3)^t - \alpha_2(2, 1)^t + \alpha_3(6, 3)^t \Rightarrow$
 $\alpha_1 = 0; \alpha_2 = \alpha_3 = \frac{1}{10}$

Feature Selection

- A classifier can also be used in feature selection.
- In an embedded method for feature selection, the features are obtained as a part of the training process.
- In the case of SVMs, at the end of the training, W is obtained.
- Let us consider a four-dimensional problem and the W learnt be $W = (0.001, -20, 10.9, 0)$
- This means that the second and third components of W are more important than the other components in that order. The second component votes in favour of the negative class and the third component favours the positive class.
- The first component of W in the example was 0.4 and the second was 0.2.

Linearly Non-Separable Case

- So far, we have assumed that the training data is linearly separable.
- What happens if the data is linearly non-separable?
- There is no feasible point if data are not linearly separable; no solution.
- Hence we can not find the optimal hyperplane by this formulation.
- We will introduce slack variables to handle the general case.



- It is possible to allow error ξ_i in classification based on the value of $W^t X + b$
- ξ_i measures extent of violation of optimal separation.

Soft Margin Formulation

- The modified constraints are

$$\begin{cases} W^t X_i + b \geq 1 - \xi_i & y_i = 1 \\ W^t X_i + b \leq -1 + \xi_i & y_i = -1 \\ \xi_i \geq 0 & \forall i \end{cases}$$

- ξ_i are slack variables in optimization. When $\xi_i > 0$, there is a margin error; when $\xi_i > 1$, X_i is wrongly classified.
- Note that $\xi_i = 0$ if there is no error corresponding to X_i .
- The optimization problem, in this case, is

$$\begin{aligned} & \text{Minimize } \frac{1}{2} || W ||^2 + C \sum_{i=1}^n \xi_i \\ & \text{subject to } y_i(W^t X_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

- Feasible solution always exists. C is a user specified constant like a regularization parameter.

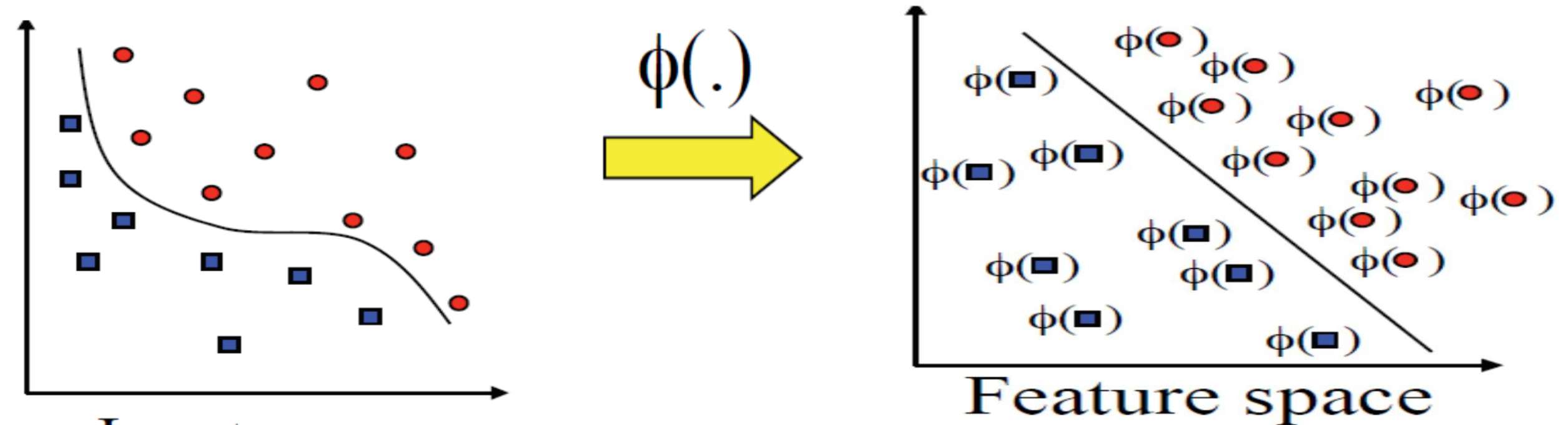
Soft Margin Formulation

- *minimize* $\frac{1}{2}W^tW + C\sum_{i=1}^n \xi_i$
subject to $1 - \xi_i - y_i(W^tX_i + b) \leq 0, i = 1, \dots, n$
 $-\xi_i \leq 0, i = 1, \dots, n$
- The Lagrangian, $\mathcal{L} = \frac{1}{2}W^tW + C\sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i(1 - \xi_i - y_i(W^tX_i + b)) - \sum_{i=1}^n \lambda_i \xi_i$
- α_i 's are lagrange multipliers for the separability constraints as earlier.
- λ_i are the lagrange multipliers for the constraints $-\xi_i \leq 0$.

$$W = \sum_{i=1}^n \alpha_i y_i X_i$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

Nonlinear Decision Boundary



Note: feature space is of higher dimension than the input space in practice

- Computational burden can be higher as the dimensionality is larger in the feature space.
- Popular scheme is to employ the kernel trick where computation is carried out in the input space.

Example Kernel Function

- We start with an example kernel function in $\mathbb{R}^2$.
- Consider $K(X_i, X_j) = (1 + X_i^t X_j)^2$.
- Let $X_i = (x_{i1}, x_{i2})^t \in \mathbb{R}^2$ and similarly for X_j .
- Then $K(X_i, X_j) = (1 + x_{i1}x_{j1} + x_{i2}x_{j2})^2$
- We now show that there exists a mapping ϕ such that $K(X_i, X_j) = \phi(X_i)^t \phi(X_j)$.

$$\phi(X_i) = (1, \sqrt{2}x_{i1}, \sqrt{2}x_{i2}, x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}x_{i2})$$

$$K(X_i, X_j) = \phi(X_i)^t \phi(X_j)$$

Kernel Trick

- Specifically, we use the kernel function, $K : \Re^m \times \Re^m \rightarrow \Re$ such that $K(X_i, X_j) = \phi(X_i)^t \phi(X_j)$.
- Let α_i be the solution of dual. Then $W = \sum \alpha_i y_i \phi(X_i)$.
- So, $b = y_j - W^t \phi(X_j) = (y_j - \sum_i \alpha_i y_i \phi(X_i)^t \phi(X_j))$
- Given a new pattern X we only need to compute
$$f(X) = W^t Z + b$$
$$= \sum_i \alpha_i y_i \phi(X_i)^t \phi(X) + b$$
$$= \sum_i \alpha_i y_i K(X_i, X) + (y_j - \sum_i \alpha_i y_i K(X_i, X_j))$$

Some Popular Kernel Functions

- Polynomial Kernel:

$$K_p(X_1, X_2) = (1 + X_1^t X_2)^p$$

- Gaussian Kernel:

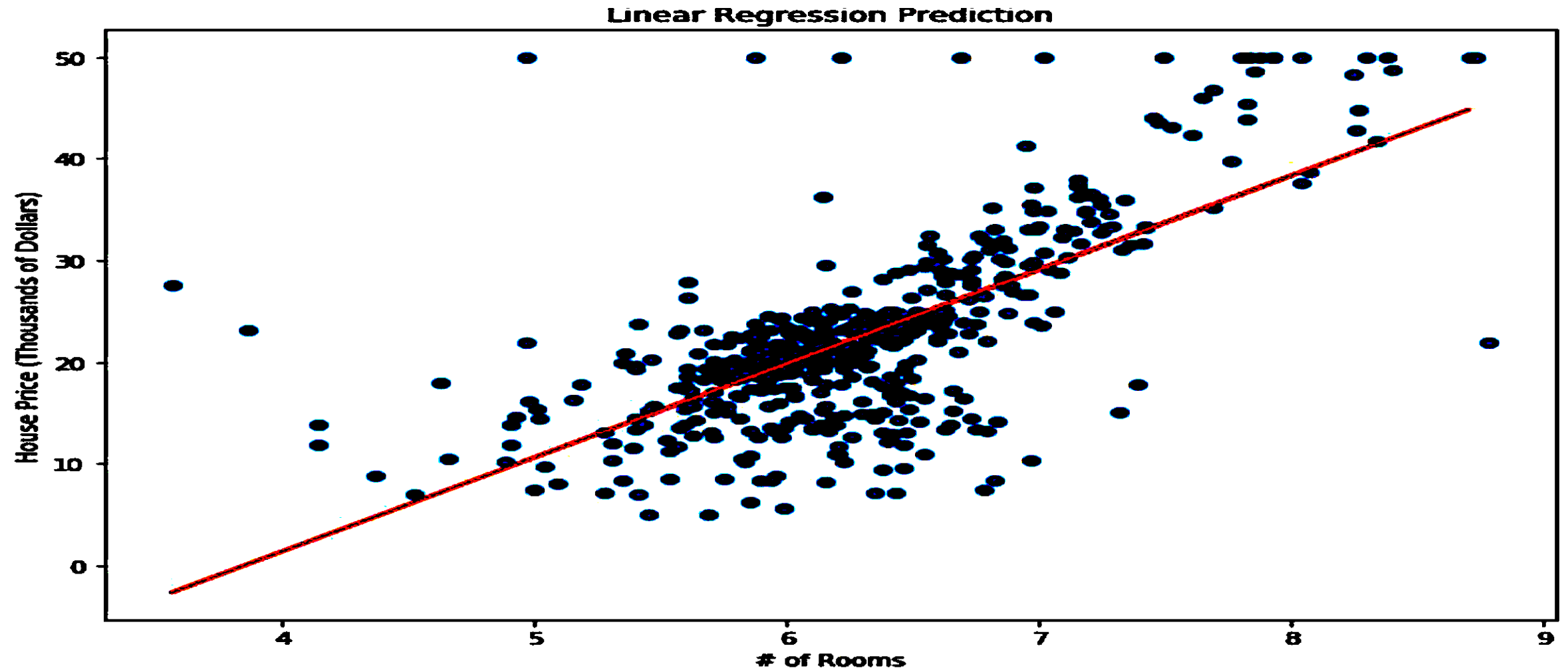
$$K_G(X_1, X_2) = e^{-\frac{\|x_1 - x_2\|^2}{\sigma^2}}$$

- Sigmoidal Kernel:

$$K_S(X_1, X_2) = \tanh(aX_1^t X_2 + \theta)$$

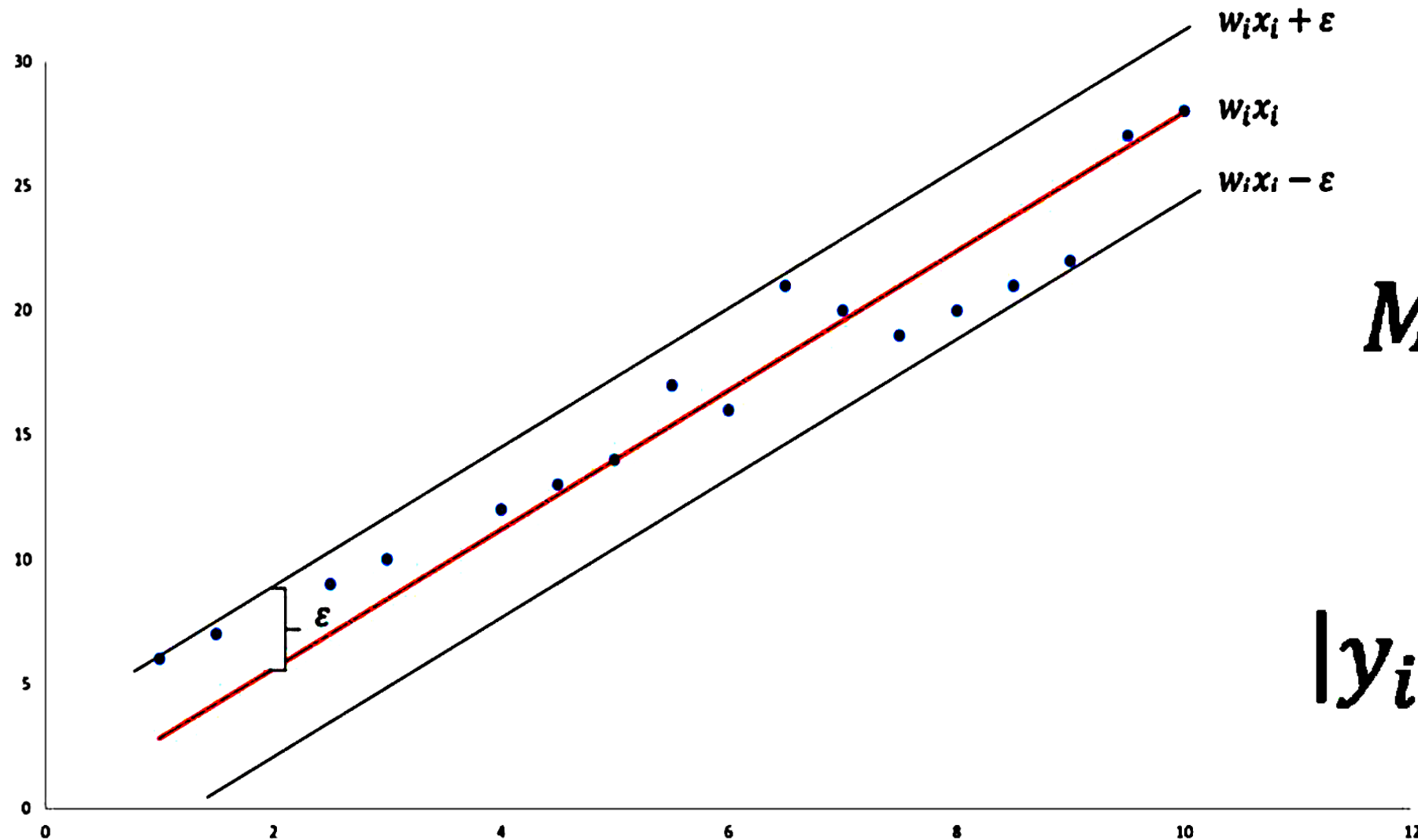
Linear Regression

$$\text{MIN} \sum_{i=1}^n (y_i - w_i x_i)^2$$



OLS Prediction of Boston Housing Prices

Support Vector Regression



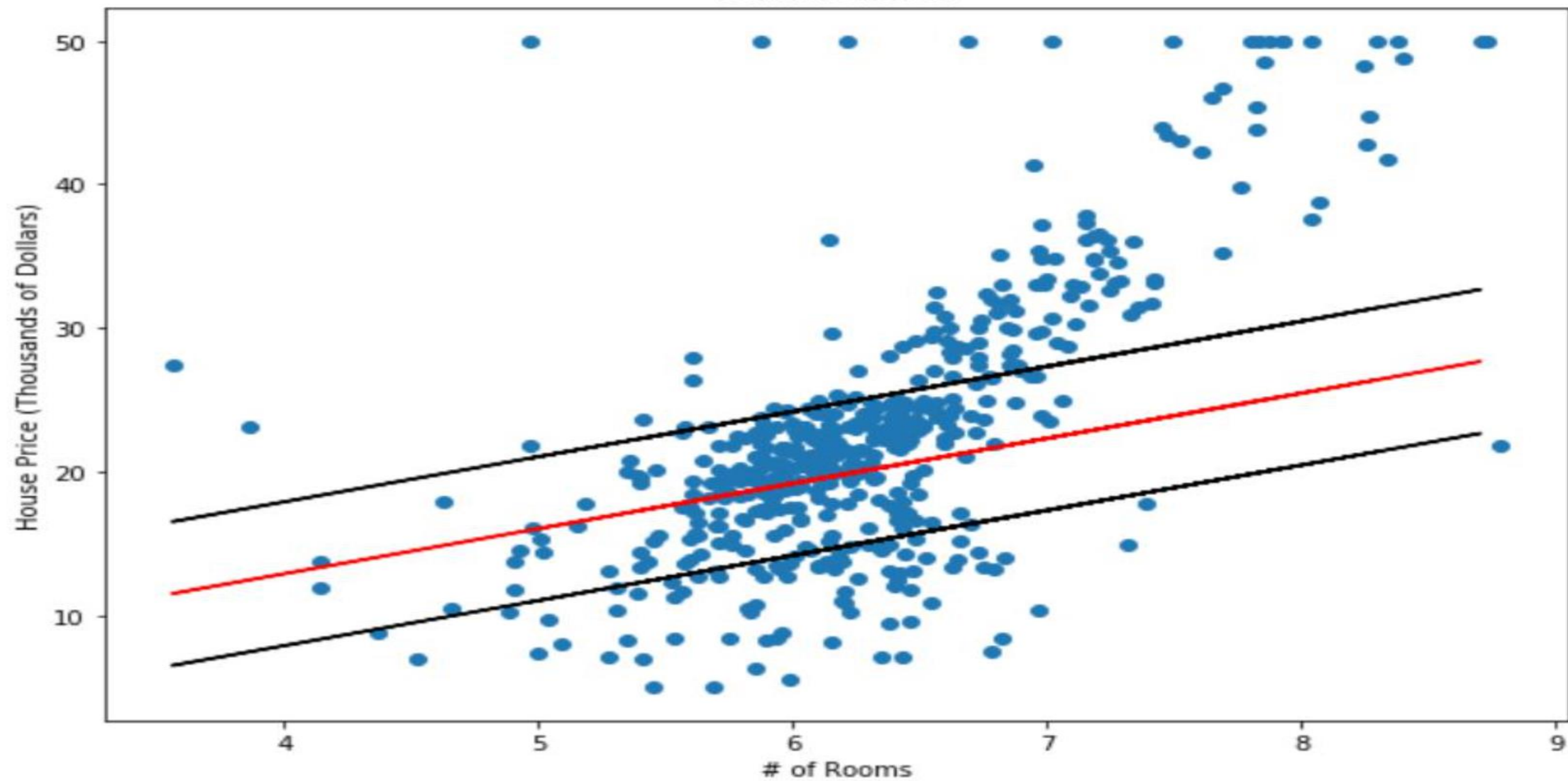
$$\text{MIN } \frac{1}{2} ||\mathbf{w}||^2$$

Such that

$$|y_i - w_i x_i| \leq \epsilon$$

Illustrative Example of Simple SVR

SVR Prediction

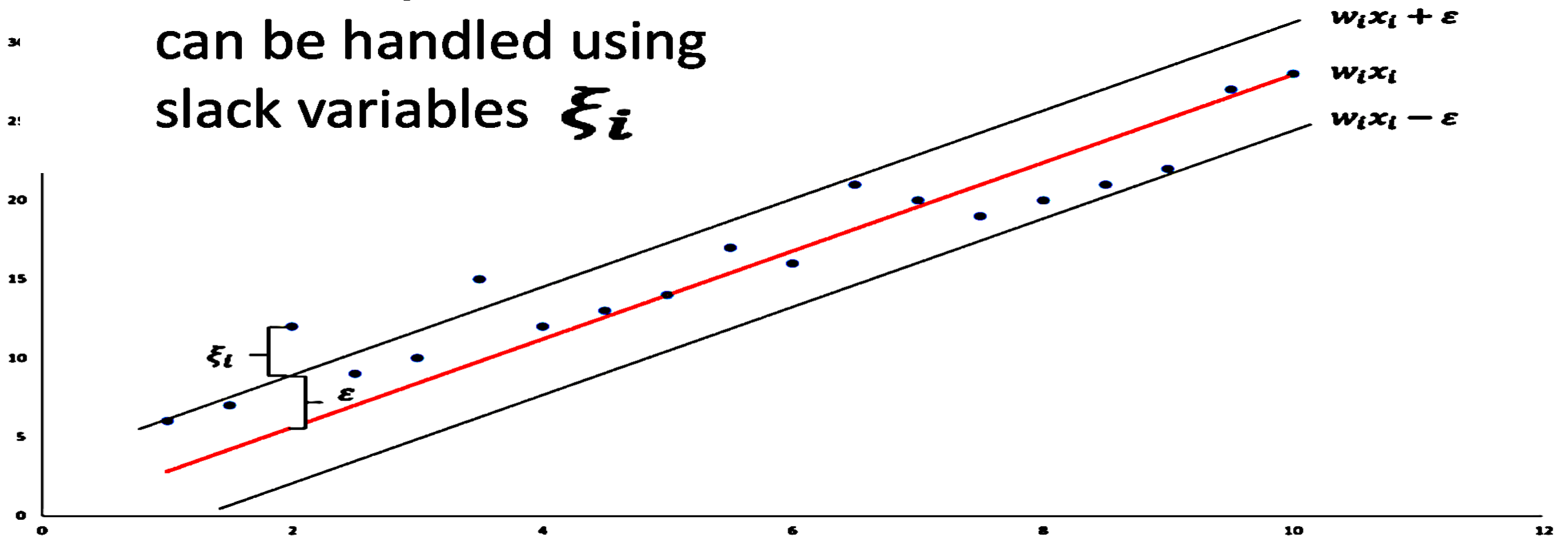


SVR Prediction of Boston Housing Prices with $\epsilon=5$

SVR with Slack Variables

$$|y_i - w_i x_i| \leq \varepsilon + |\xi_i| \quad \text{MIN} \frac{1}{2} ||w||^2 + C \sum_{i=1}^n |\xi_i|$$

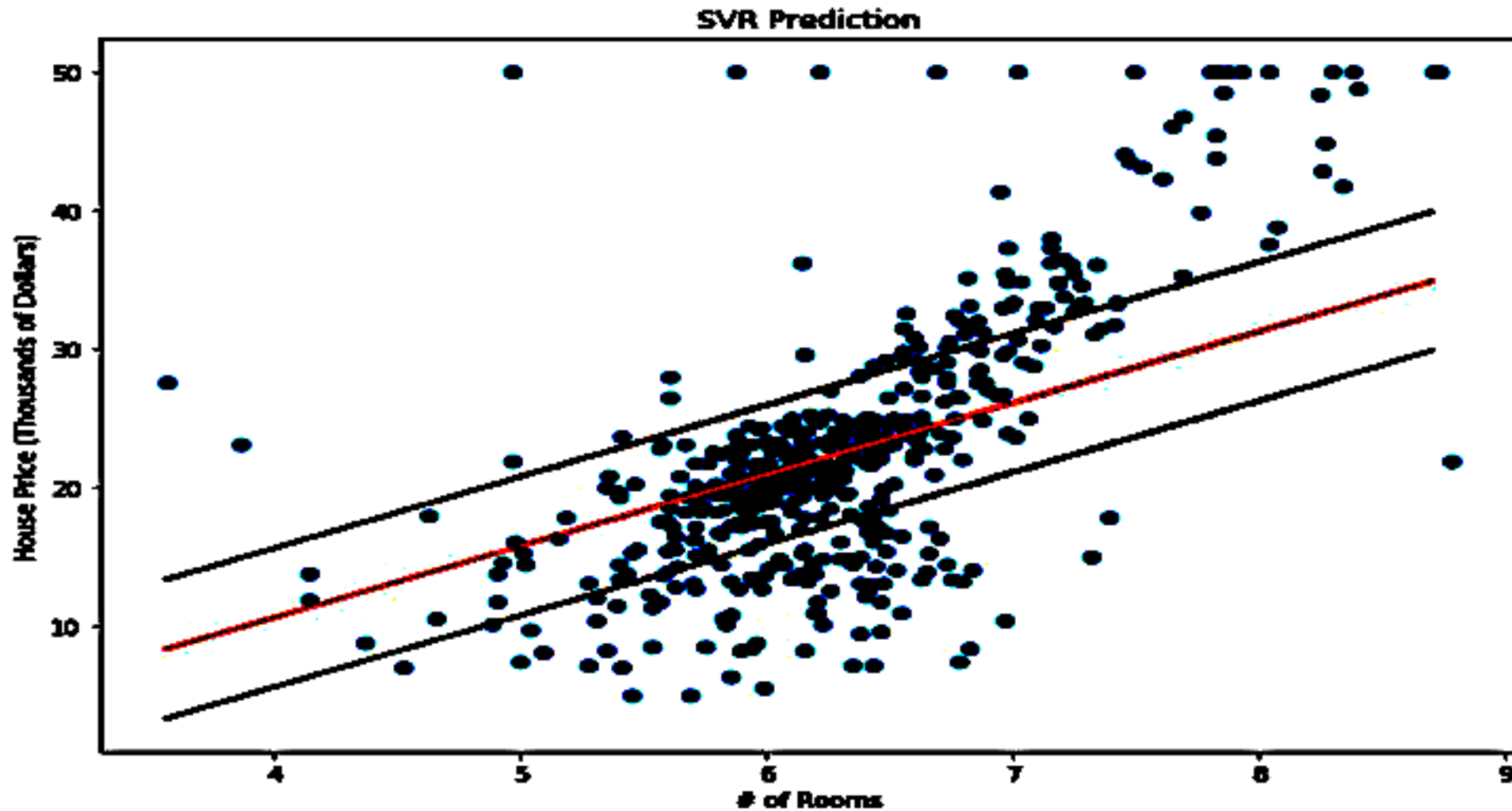
Errors larger than ε
can be handled using
slack variables ξ_i



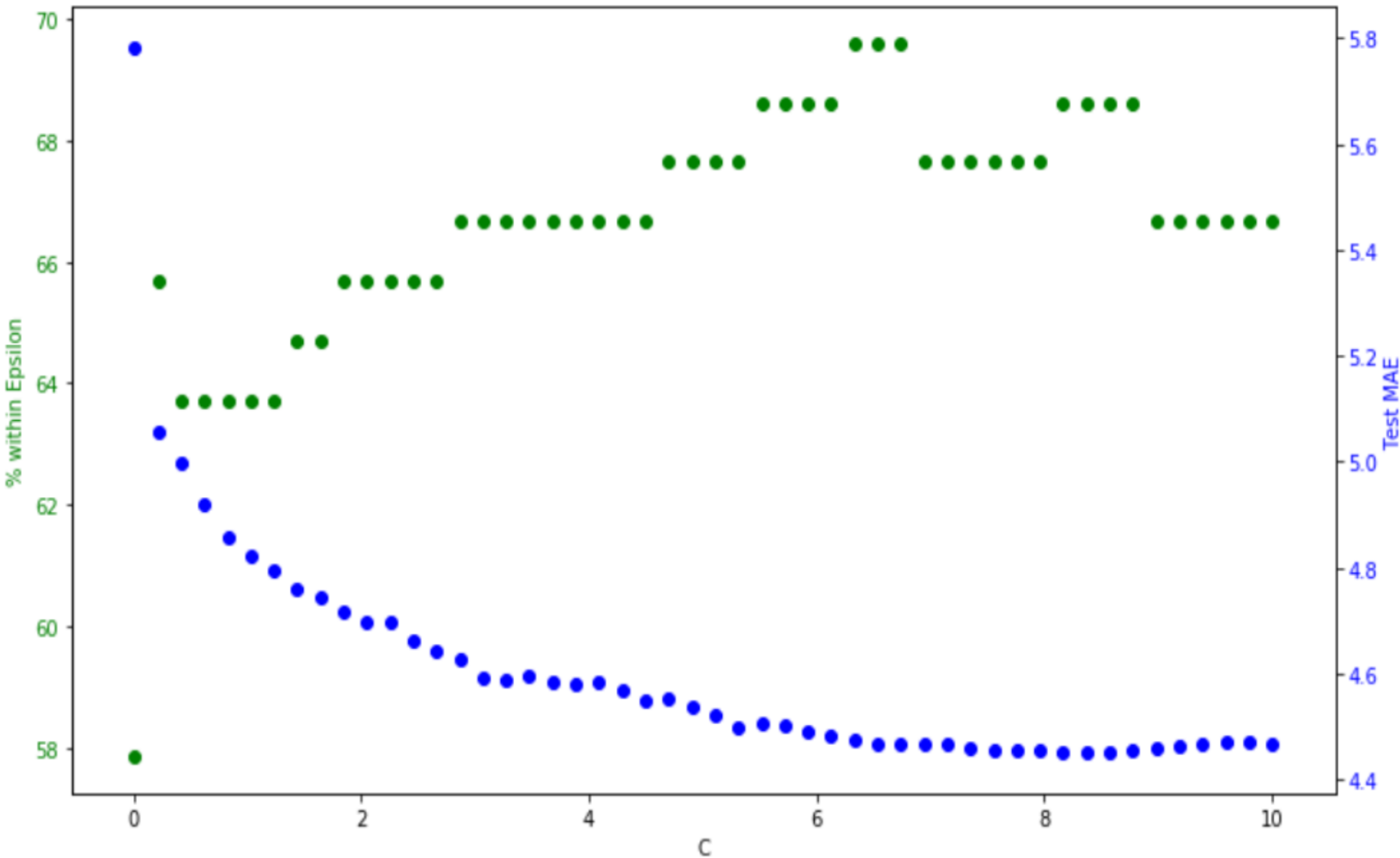
Illustrative Example of SVR with Slack Variables

SVR with Slack Variables

Let's set $C=1.0$ and retrain our model above. The results are plotted below:



Searching for Optimal C



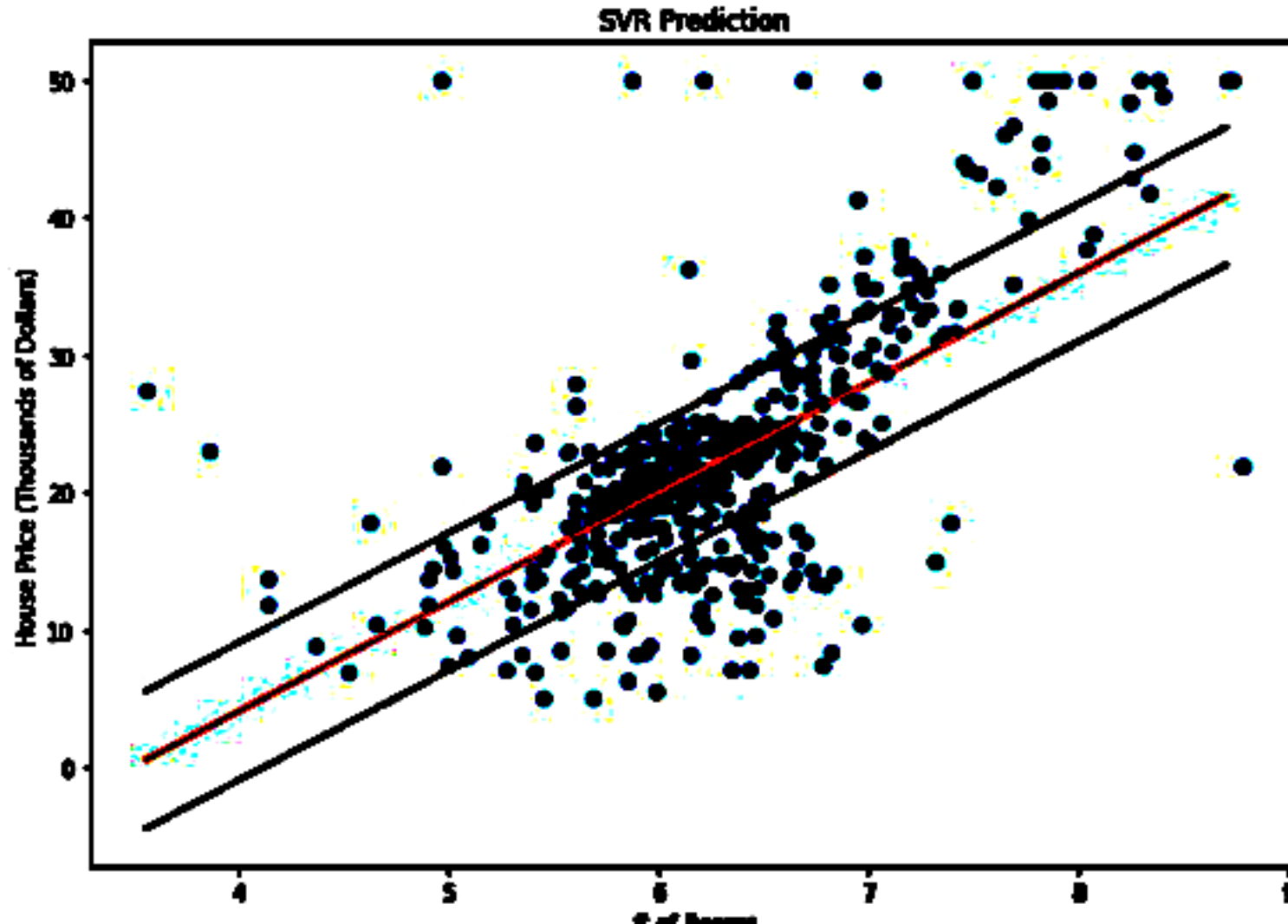
Mean Absolute Error

Find C to maximize %
Within ϵ

Optimal C = 6.1

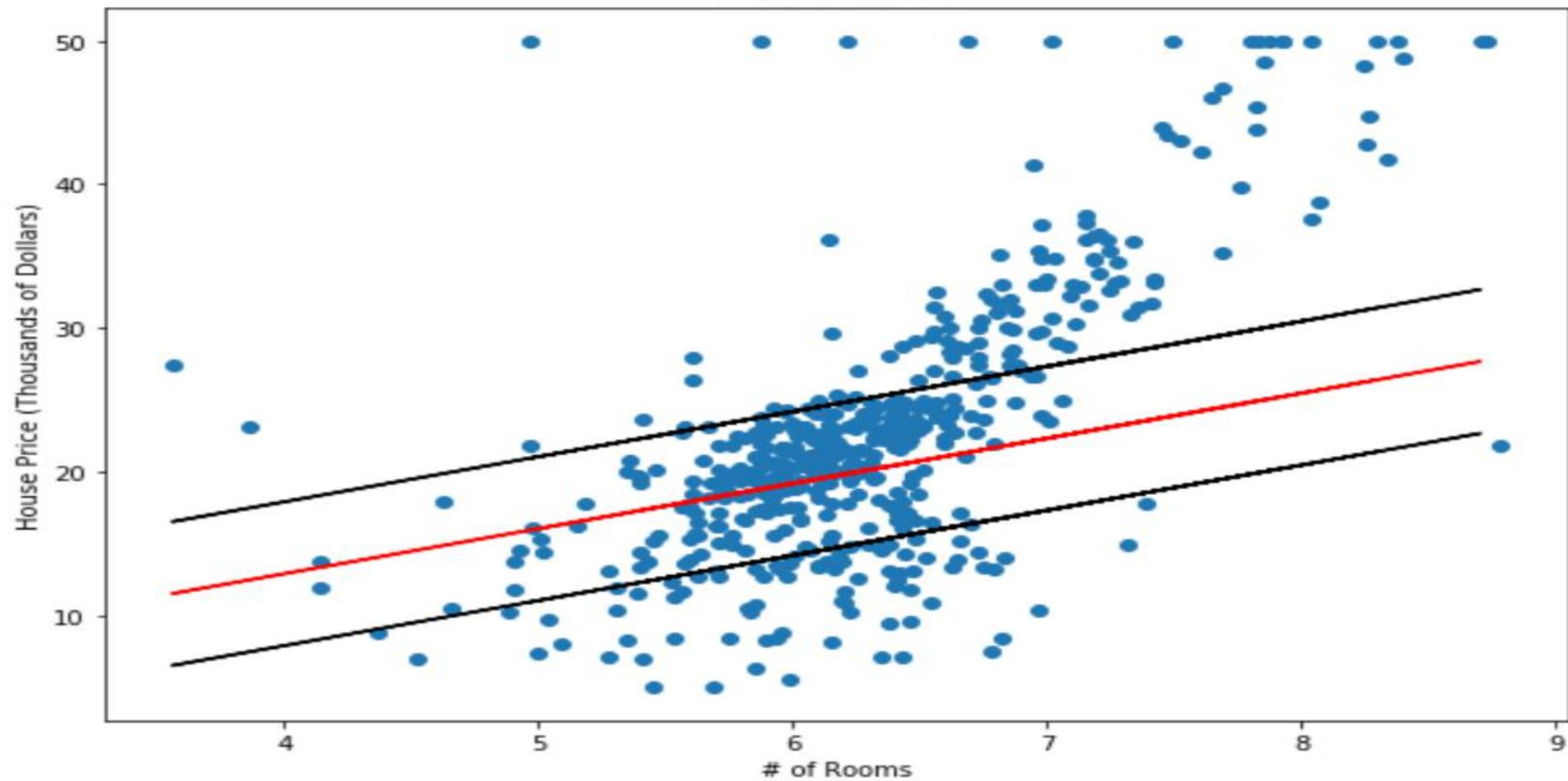
SVR with Optimal C

Let's build one last model with our final hyperparameters, $\epsilon=5$, $C=6.13$.



SVR combines tolerance of errors through an error margin ϵ and additional tolerance through ξ_i

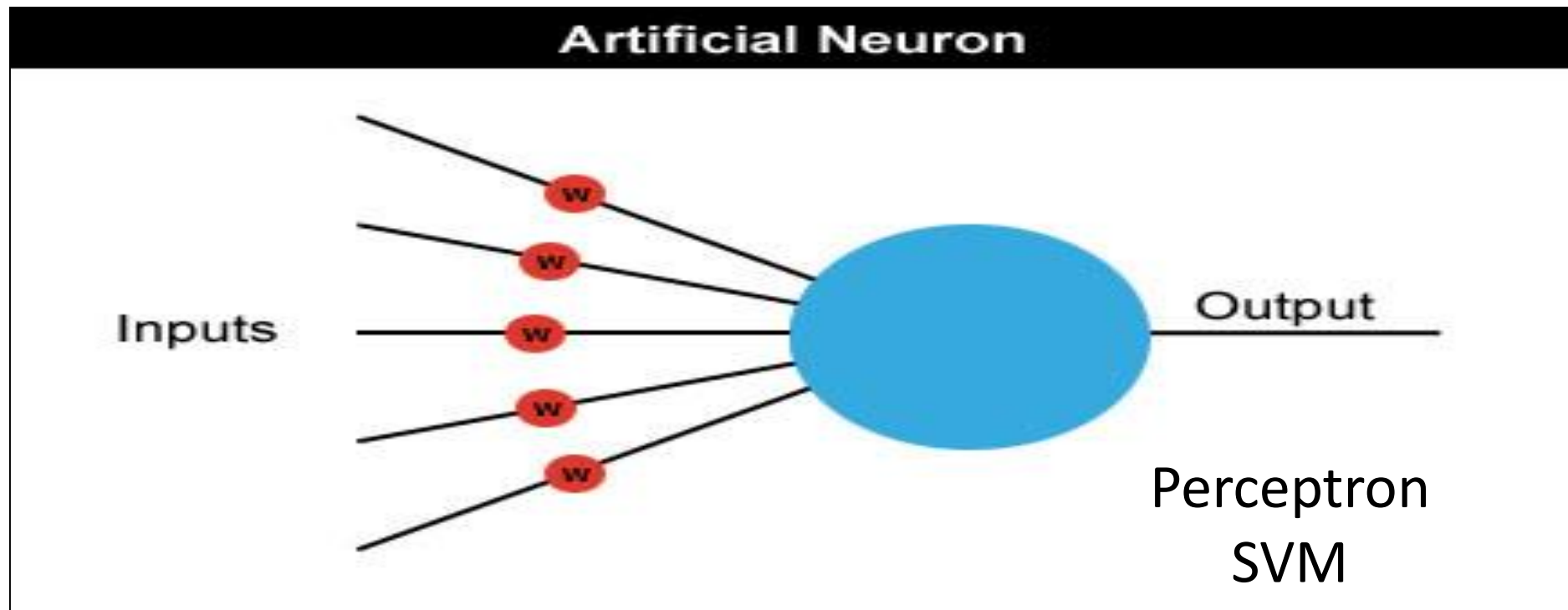
SVR Prediction



SVR Prediction of Boston Housing Prices with $\epsilon=5$

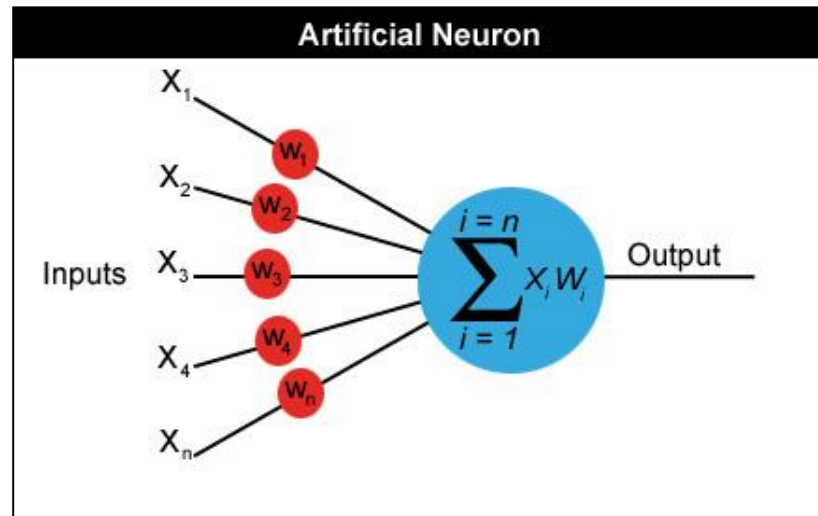
Neural Networks

- Neural networks connect many artificial neurons.
- Each input into the neuron has its own weight associated with it illustrated by the red circle.
- Each weight is a real number and we adjust the weight during training of the network.



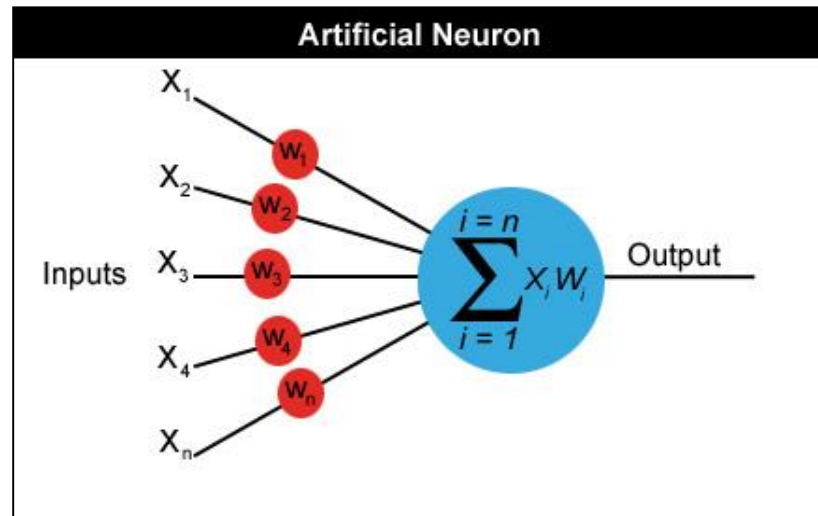
Neural Networks

- A neuron can have any number of inputs from one to d , where d is the total number of inputs.
- The inputs may be represented as $x_1, x_2, \dots, x_d$.
- The corresponding weights for the inputs are $w_1, w_2, \dots, w_d$.
- Output $a = w_1x_1 + w_2x_2 + \dots + w_dx_d$



Artificial Neural Networks

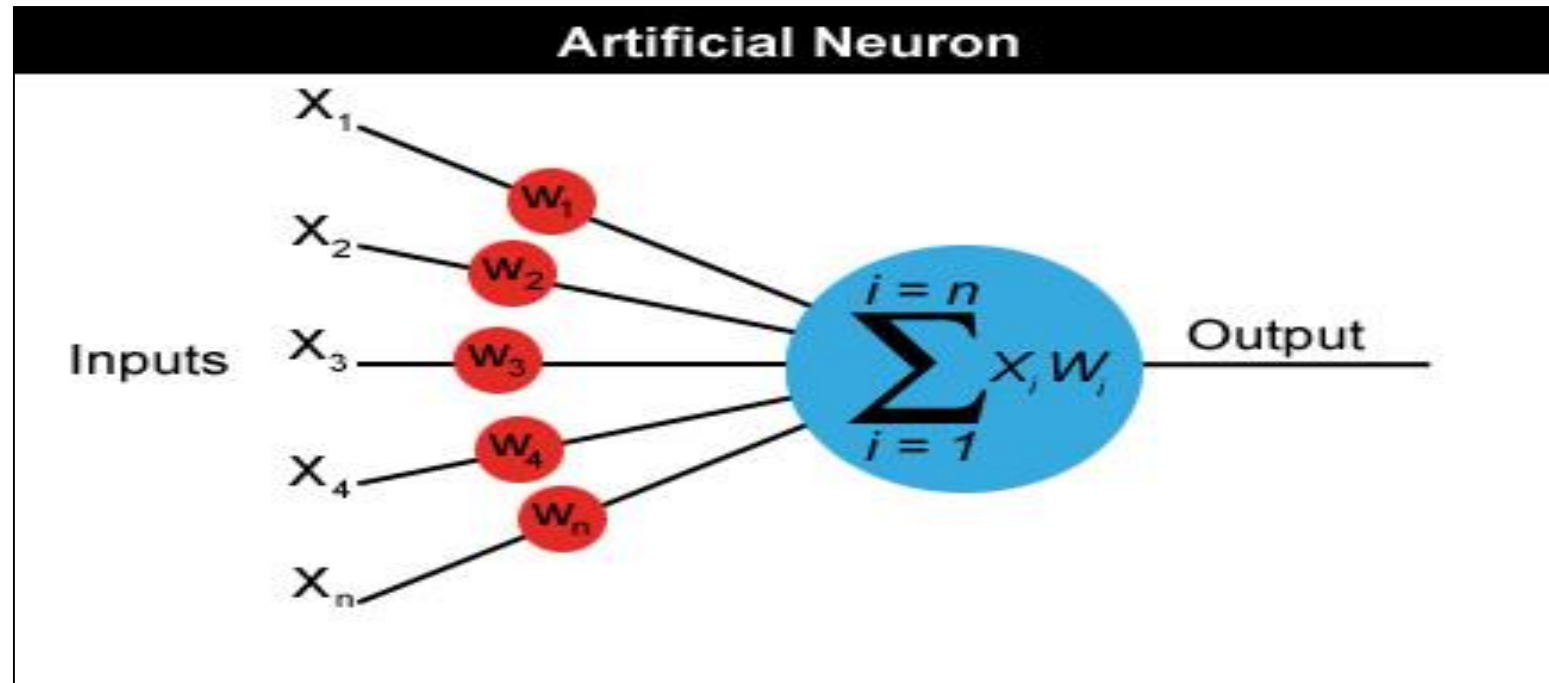
- The inputs represented as $x_1, x_2, \dots, x_d$.
- The corresponding weights being $w_1, w_2, \dots, w_d$.
- Output $a = w_1x_1 + w_2x_2 + \dots + w_dx_d$
- We have seen Perceptron and linear SVM



Linear Models: Limits

We can give each feature a weight

- But not more complex value relationships, e.g,
 - any value in the range [0;4] is good – values over 4 are bad and higher than 9 is good.

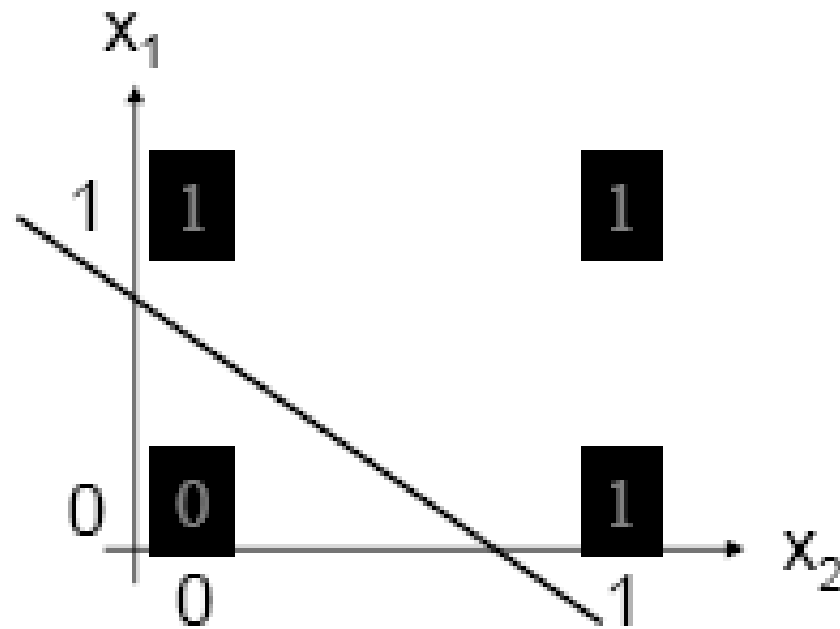


Perceptron Learning

- A perceptron (threshold unit) can *learn* anything that it can *represent* (i.e. anything separable with a hyperplane)

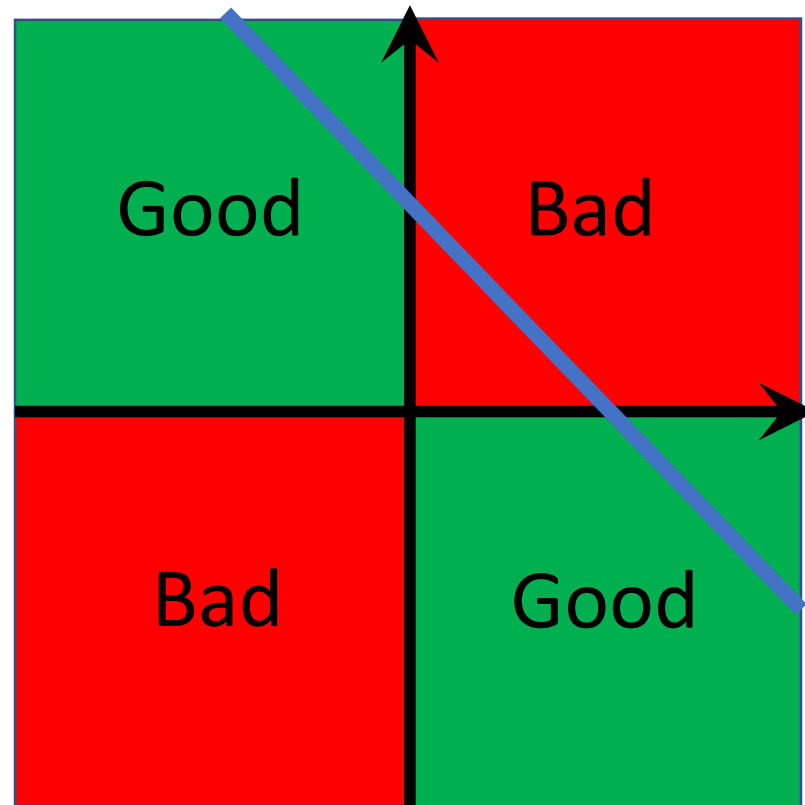
OR function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



XOR Problem

Models linear in the inputs cannot model XOR



The Exclusive OR (XOR) Problem

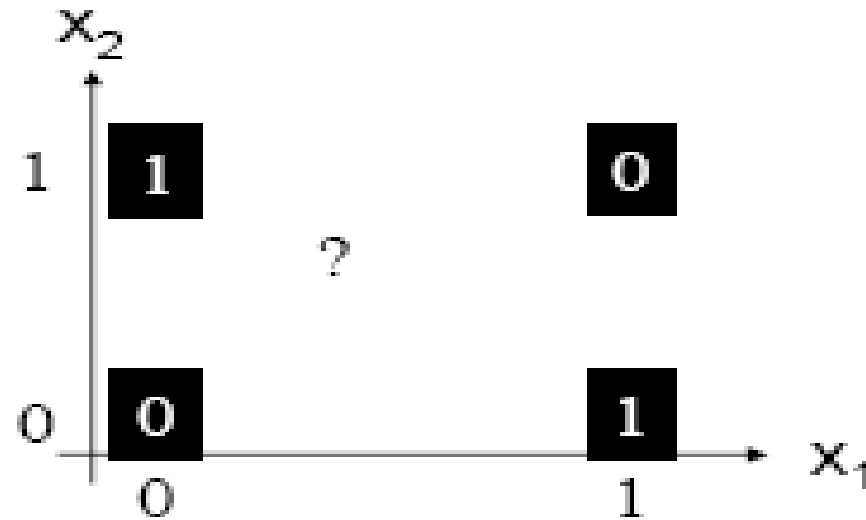
A Perceptron cannot represent Exclusive OR since it is not linearly separable.

(WRONG)

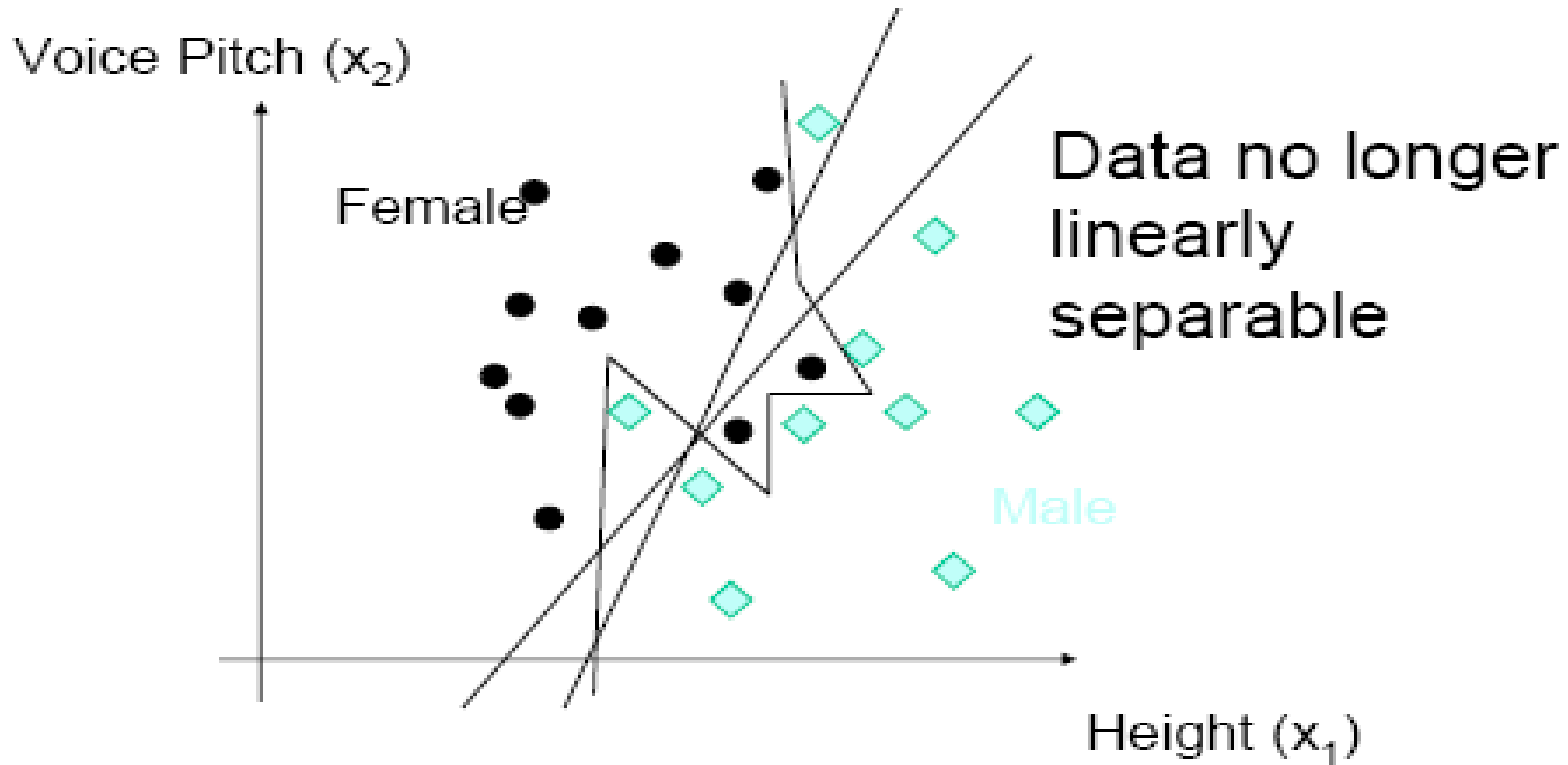
A perceptron cannot represent XOR in the input space.

XOR function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Non-Linear Decision Boundary



How to deal with such a decision boundary?
Is there a way to say which decision boundary is good?

Learning with Hidden Units

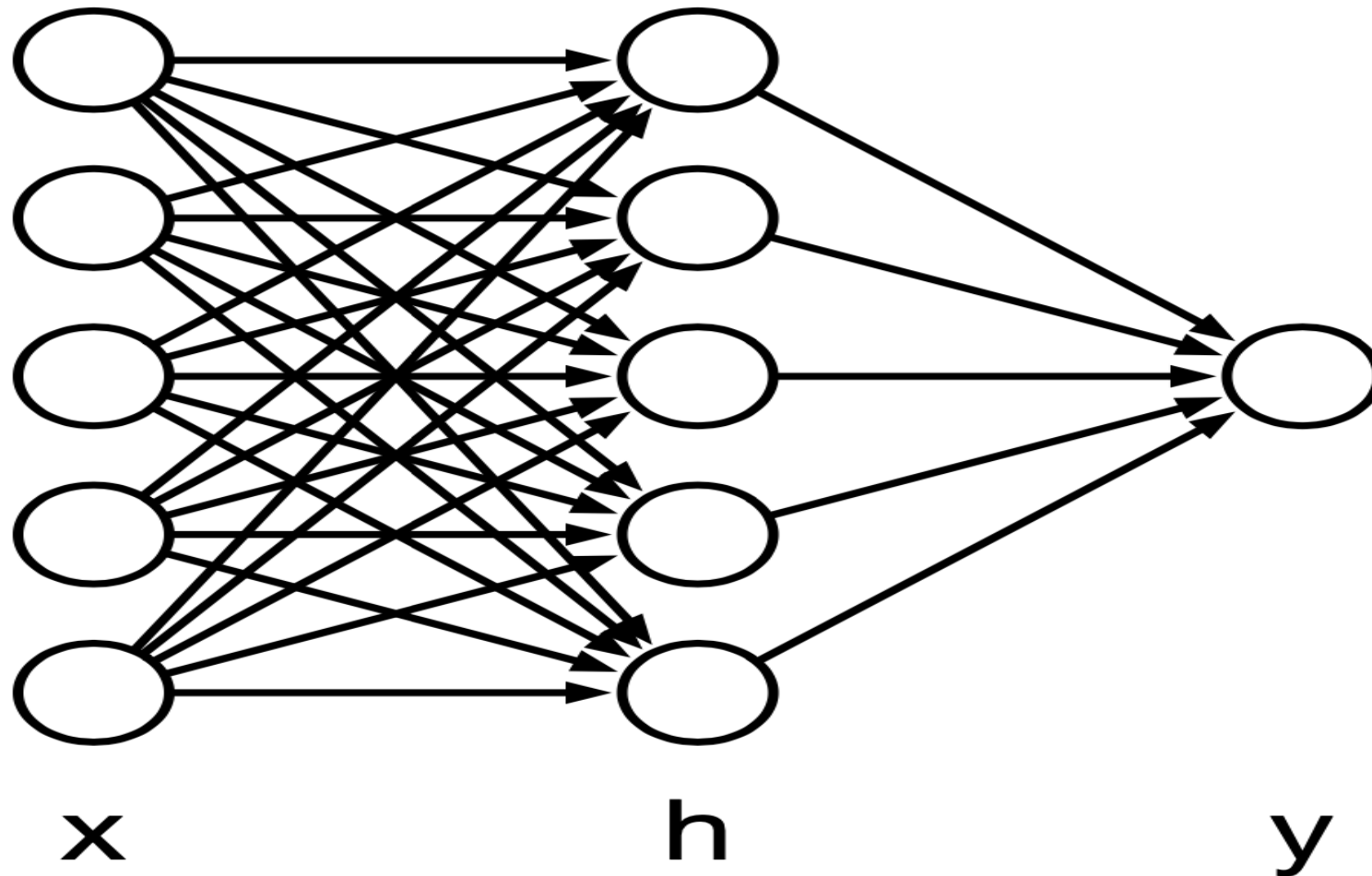
- Networks without hidden units are very limited in the input-output mappings they can model.
- Adding a layer of hand-coded features (as in a perceptron) makes them much more powerful but the hard bit is designing the features.
 - We would like to find good features without requiring insights into the task or repeated trial and error where we guess some features and see how well they work.
- We need to automate the loop of designing features for a particular task and seeing how well they work.

Solutions?

1. Find out a better set of features and use
 - It could be based on domain knowledge
 - Feature selection
 - Feature extraction
2. One way of automatically doing this is to use additional layers of neurons
 - Such a network is called a Multi-Layer Perceptron (MLP) network

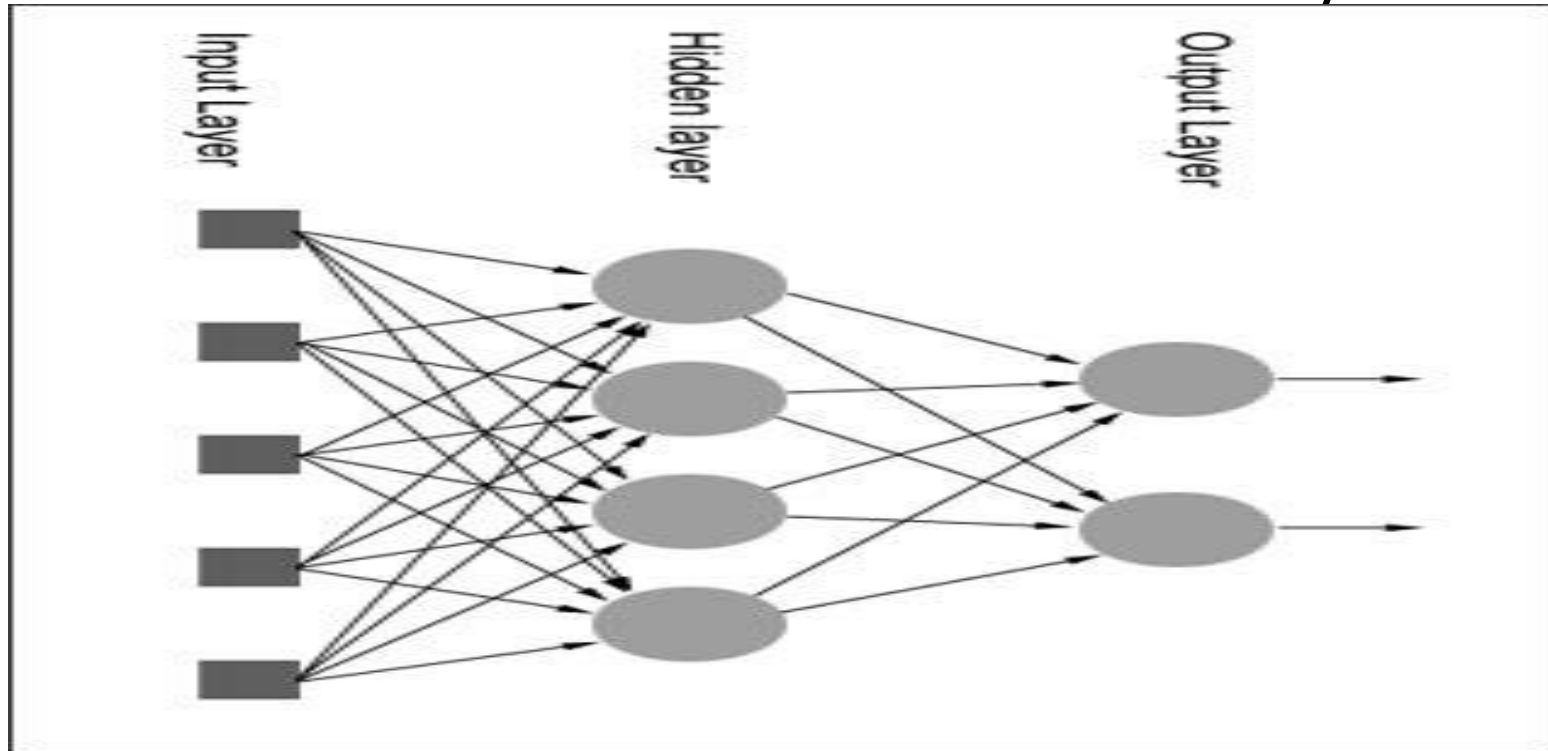
Multiple Layers

Add an intermediate (hidden) layer of processing units (each arrow is a weight)



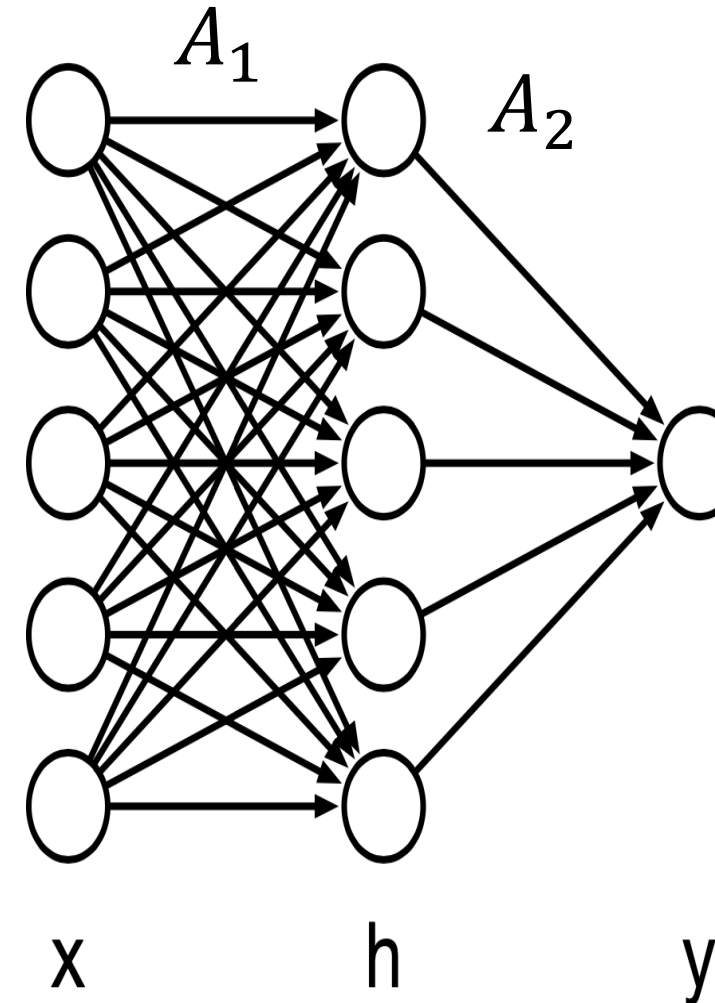
What is an MLP?

- *Feedforward network*: The neurons in each layer feed their output forward to the next layer until we get the final output from the neural network.
- There can be many hidden layers within a feedforward network.
- The number of neurons can be fixed liberally.



One Hidden Layer

- Step function or linear threshold activation will not work as it is not differentiable
- Using a linear activation function throughout the MLP will amount effectively to a single layer network
 - For Example, If $Y_1 = A_1 X$ and $Y_2 = A_2 Y_1$, then $Y_2 = A_2 A_1 X$.
 - Note that it is of the form $Y_2 = B X$, where $B = A_2 A_1$
 - So, it is effectively a single-layer computation.



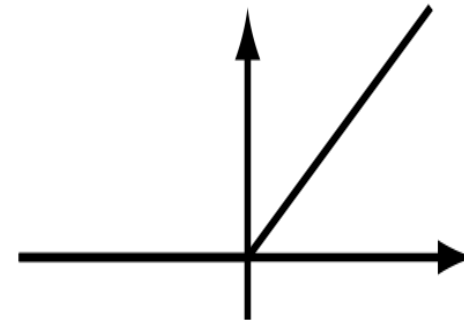
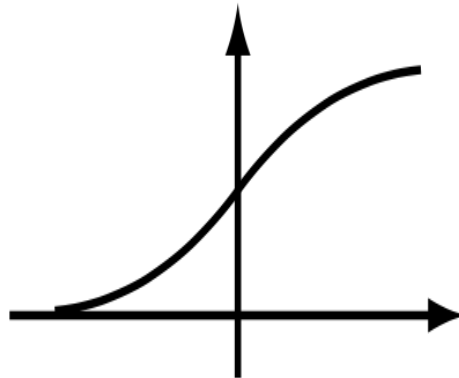
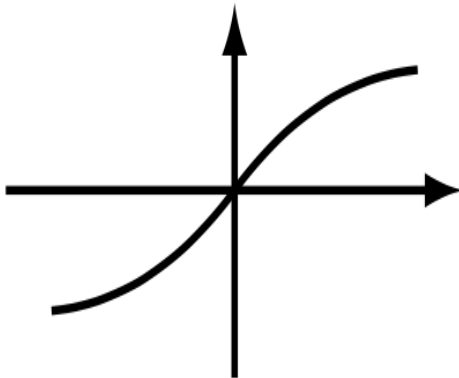
Non-Linear Activation Functions

- Instead of $W^t X$ which is used by the linear activation units, we have
- Output = $f(W^t X)$ where f is a non-linear activation function.
- Popular choices for f are:

$$\tanh(x)$$

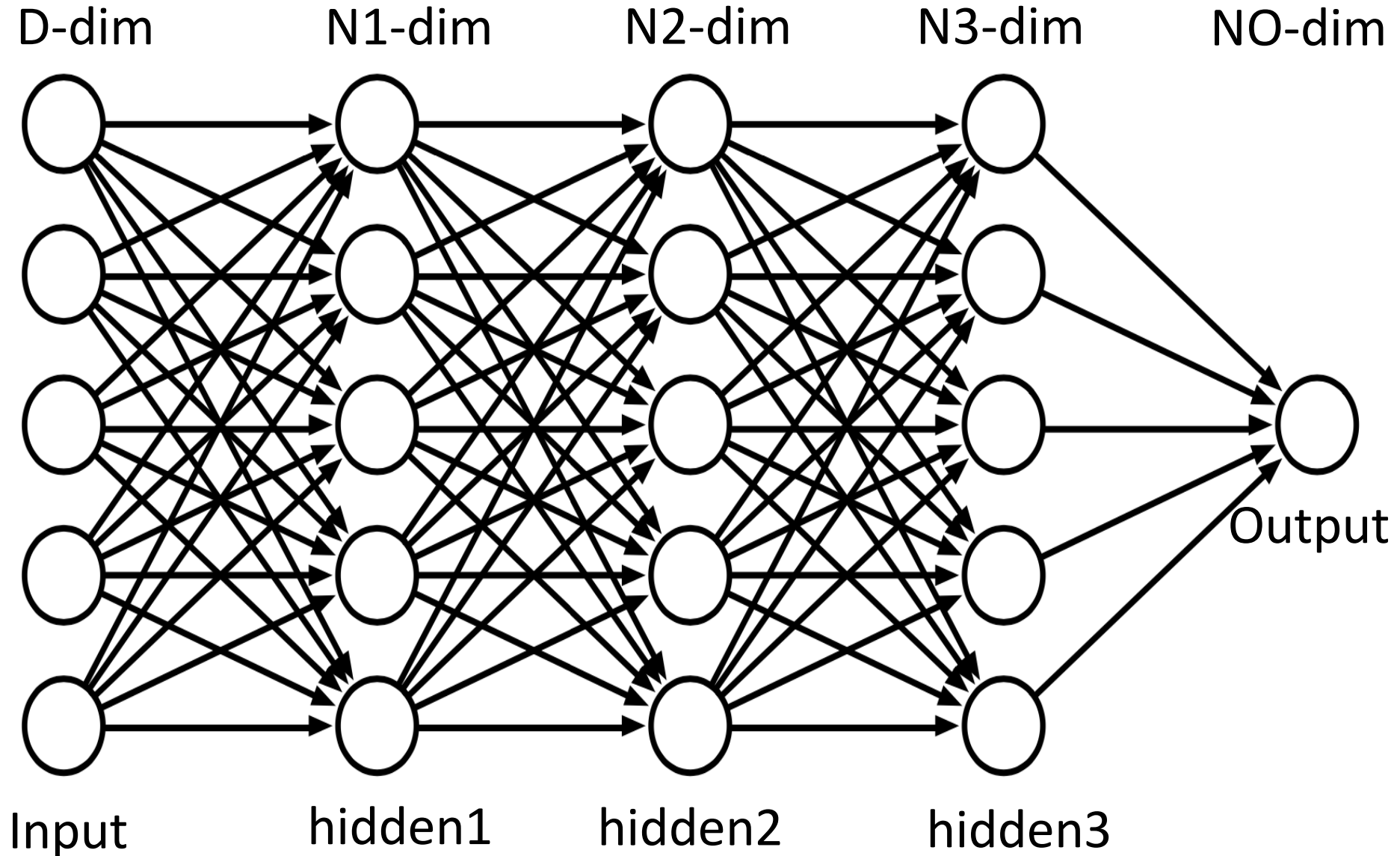
$$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

$$\text{relu}(x) = \max(0, x)$$



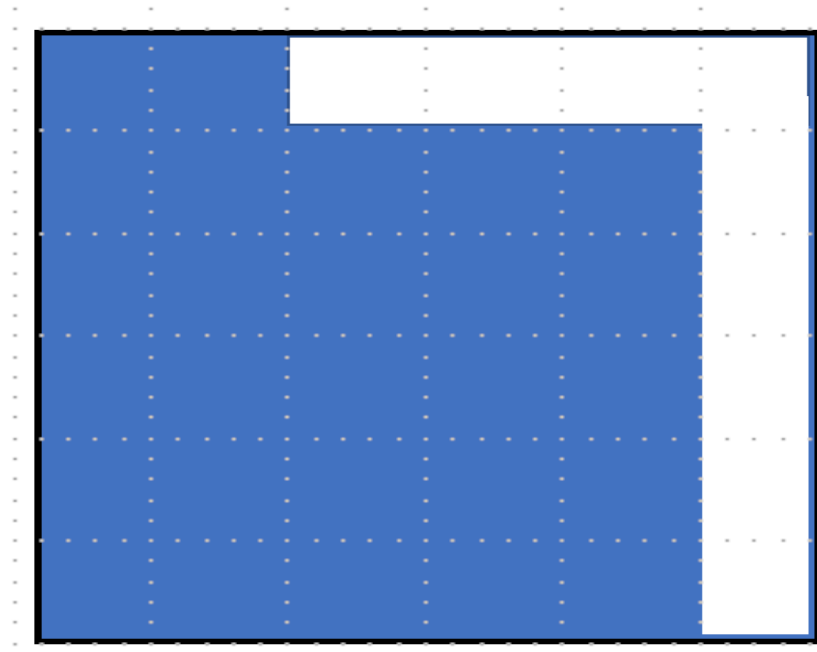
Sigmoid is also called the logistic function

Deep Learning



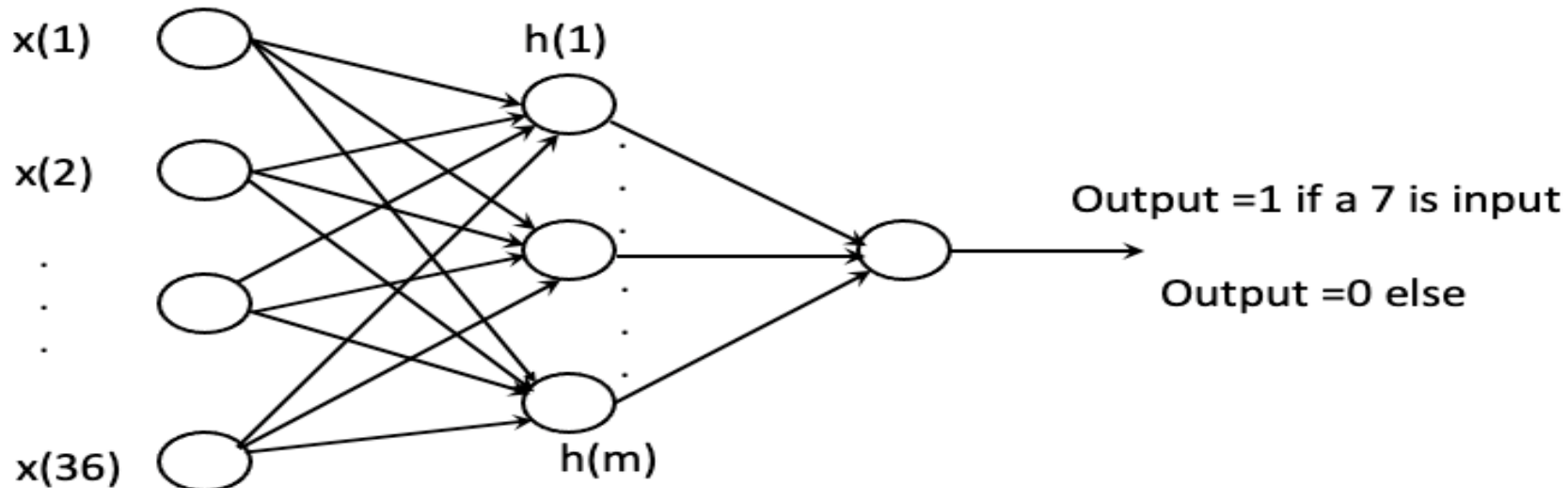
Neural Networks: Example

- A neural network that will detect the number '7'.
- Neural net to classify the input as character '7' or not '7'.
- The input is an six cells square and looks like this:
- The net will have **36 inputs**, each one representing a particular cell in the input and a hidden layer consisting of a number of neurons all feeding their output into just **one neuron in the output** layer



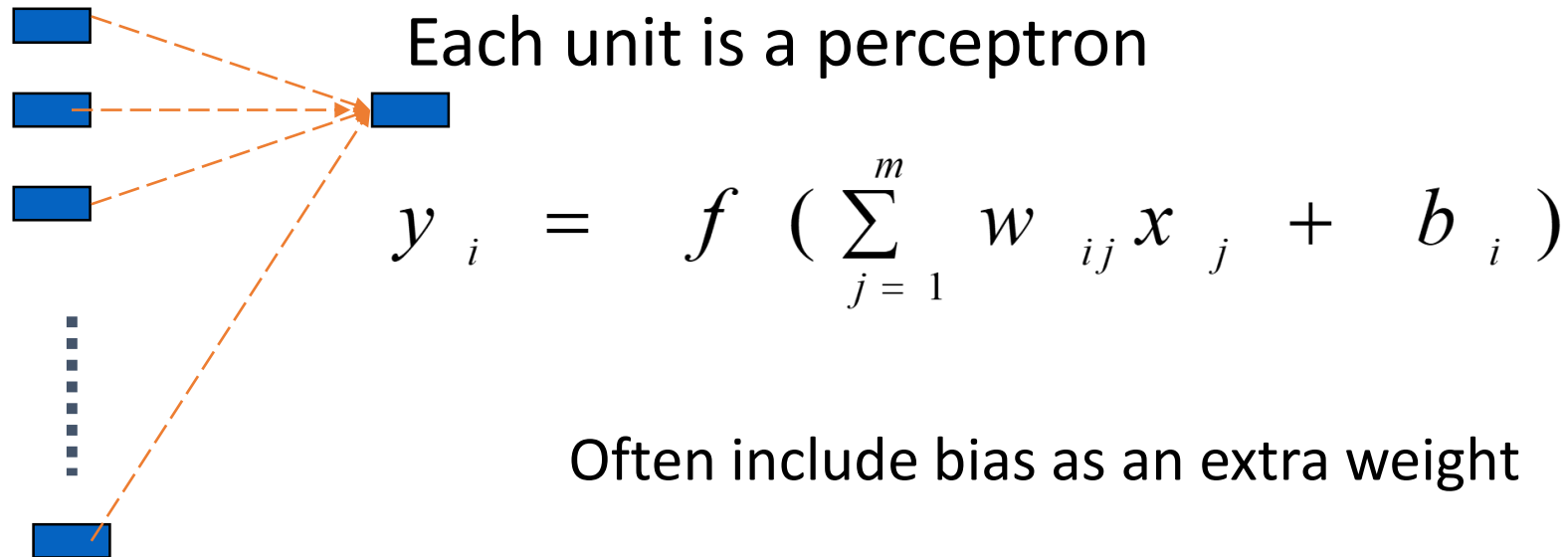
Neural Networks: Example

- initialize the neural net with random weights
- feed it a series of inputs which represent, in this example, the different 6x6 binary configurations
- for each configuration we get the output and **adjust the weights accordingly** so that when it sees a character 7 it outputs a 1 and for everything else it outputs a zero.

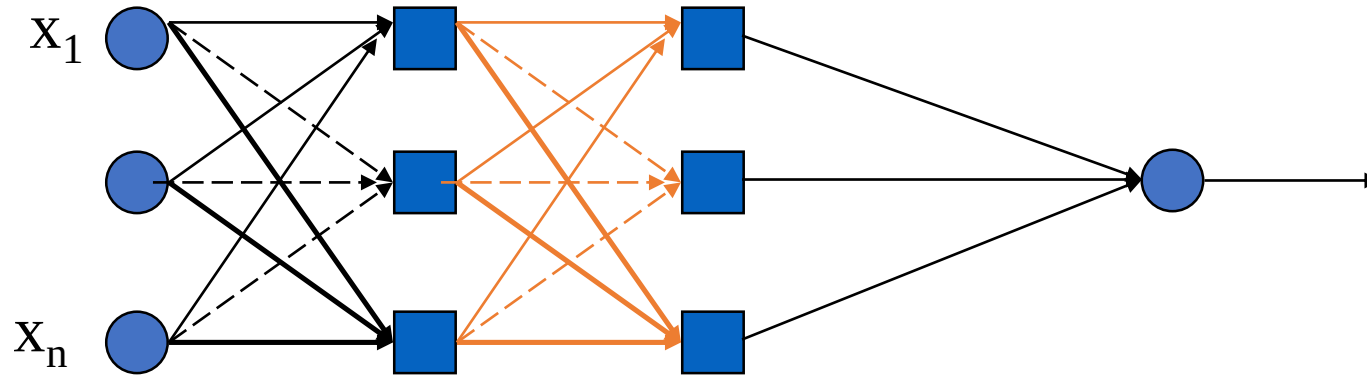


MLP Architecture

- No connections within a layer
- No direct connections between input and output layers
- Fully connected between layers
- Often more than 3 layers
- Number of output units need not equal number of input units
- Number of hidden units per layer can be more or less than input or output units



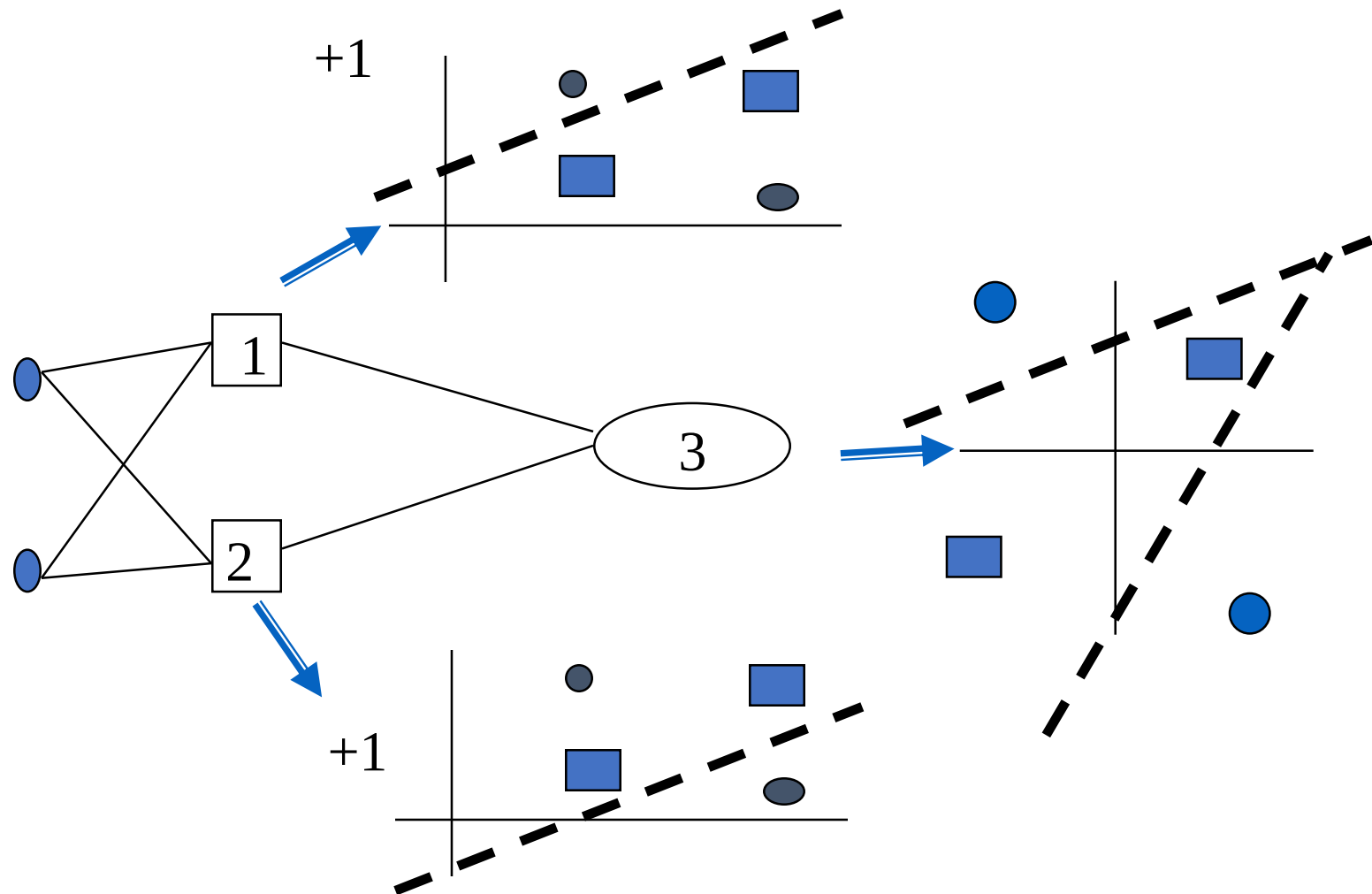
MLP: Number of Layers



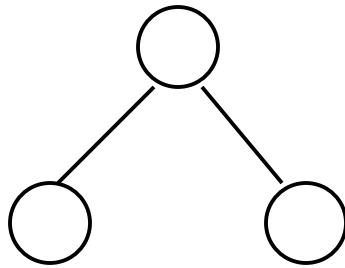
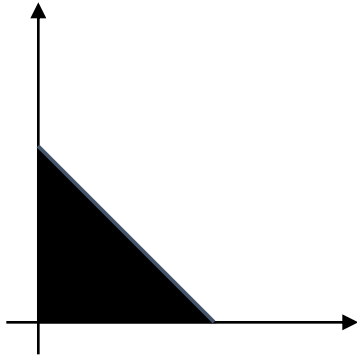
Different people refer to the above as either 4 layer (no. of layers of neurons) or 3 layer (no. of layers of adaptive weights). We will follow the latter convention

what do the extra layers give you? Start with looking at what a single layer can't do

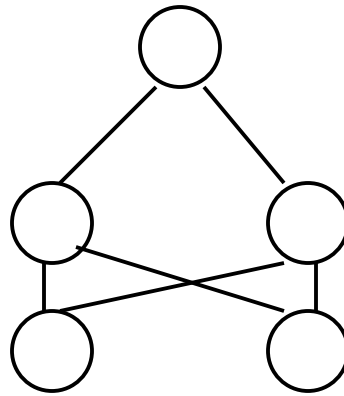
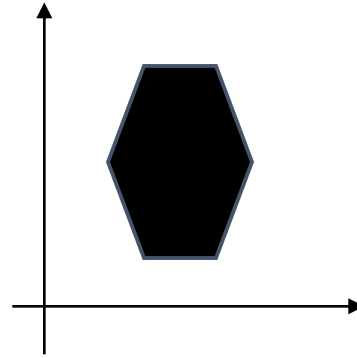
Piecewise Linear: Representation Learning



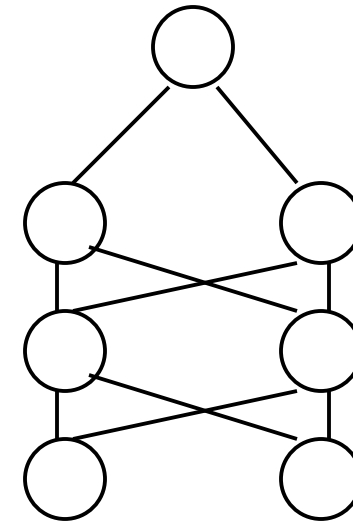
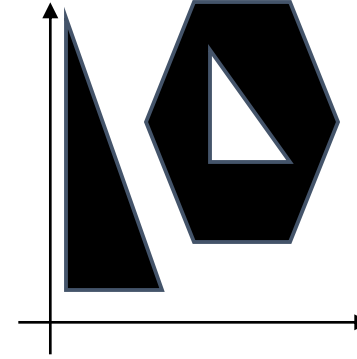
Roles of Multiple Layers



1st layer draws
linear
boundaries



2nd layer combines
the boundaries to get
a piecewise linear
region



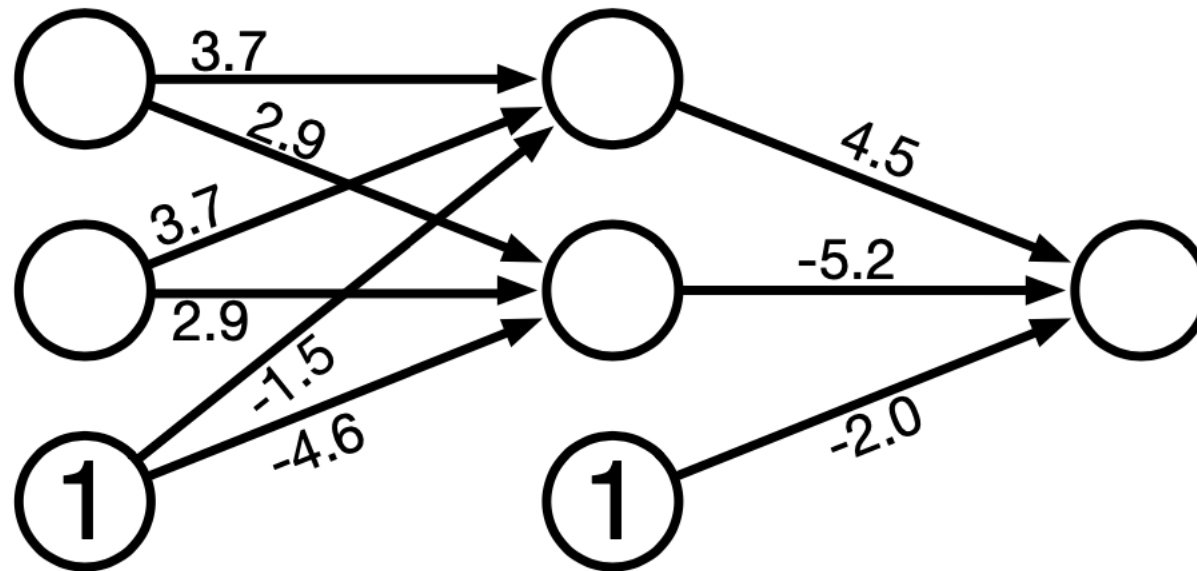
3rd layer can
generate arbitrarily
complex boundaries

How does Depth Help?

- Each layer may be viewed as a processing step
- Having multiple processing steps allows complex functions
- NN and computing circuits
 - computer = sequence of Boolean gates
 - neural computer = sequence of layers
- Deep neural networks can implement complex functions
 - For example, handwritten digit recognition

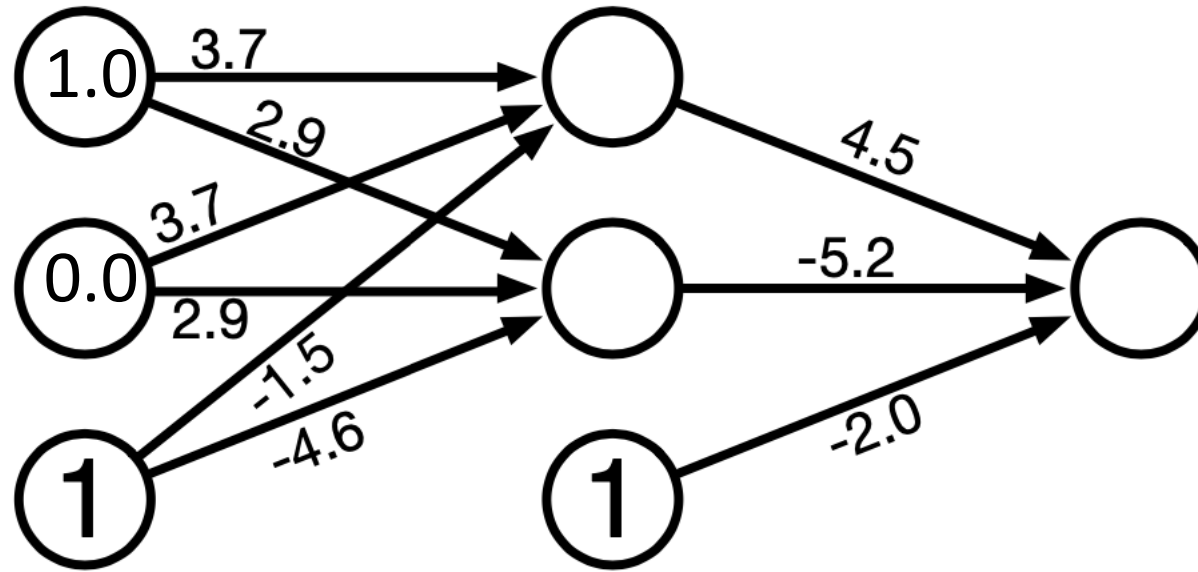
Example Neural Network

- Each layer may be viewed as a feature extractor



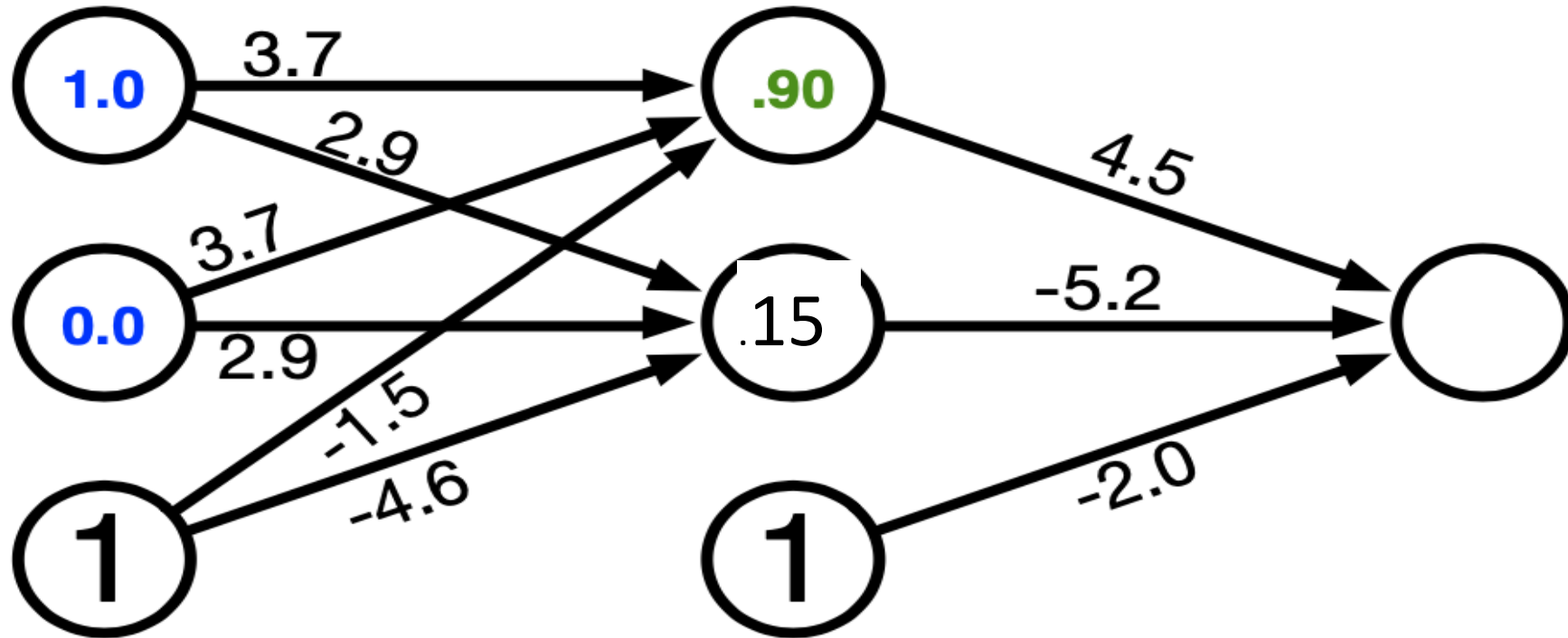
- Each bias unit is assumed to be a unit with input 1.

Forward Computation



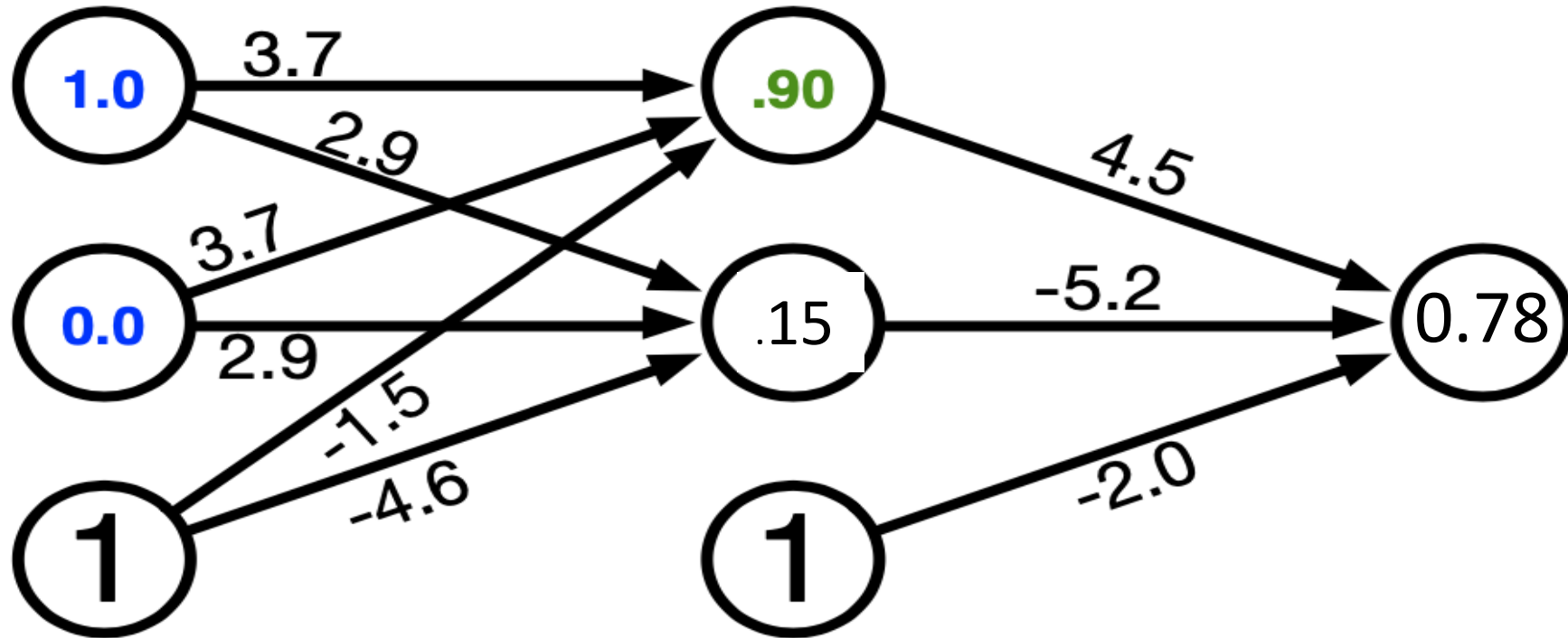
- Hidden unit outputs:
- $\text{sigmoid}(1.0 \times 3.7 + 0.0 \times 3.7 + 1 \times -1.5) =$
 $\text{sigmoid}(2.2) = \frac{1}{1 + e^{-2.2}} = 0.90$
- $\text{sigmoid}(1.0 \times 2.9 + 0.0 \times 2.9 + 1 \times -4.6) =$
 $\text{sigmoid}(-1.7) = \frac{1}{1 + e^{1.7}} = 0.17$

Output Computation



- Output: $\text{sigmoid}(.90 \times 4.5 + .15 \times -5.2 + 1 \times -2.0) = \text{sigmoid}(1.27) = \frac{1}{1 + e^{-1.27}} = 0.78$

Output Computation



- Output: $\text{sigmoid}(.90 \times 4.5 + .15 \times -5.2 + 1 \times -2.0) = \text{sigmoid}(1.27) = \frac{1}{1 + e^{-1.17}} = 0.78$
- The forward pass ends here with the output value being 0.78

Binary Input Vectors

Input x0	Input x1	Hidden1	Hidden2	Output
0	0	0.18	0.01	0.22 → 0
0	1	0.9	0.15	0.78 → 1
1	0	0.9	0.15	0.78 → 1
1	1	0.99	0.77	0.17 → 0

- Network implements XOR.
 - Hidden1 output is OR
 - Hidden2 : node AND
 - final layer operation is Hidden1- Hidden2
- Power of deep neural networks: chaining of processing steps just as: more Boolean circuits !
more complex computations possible

Backpropagation Learning Algorithm

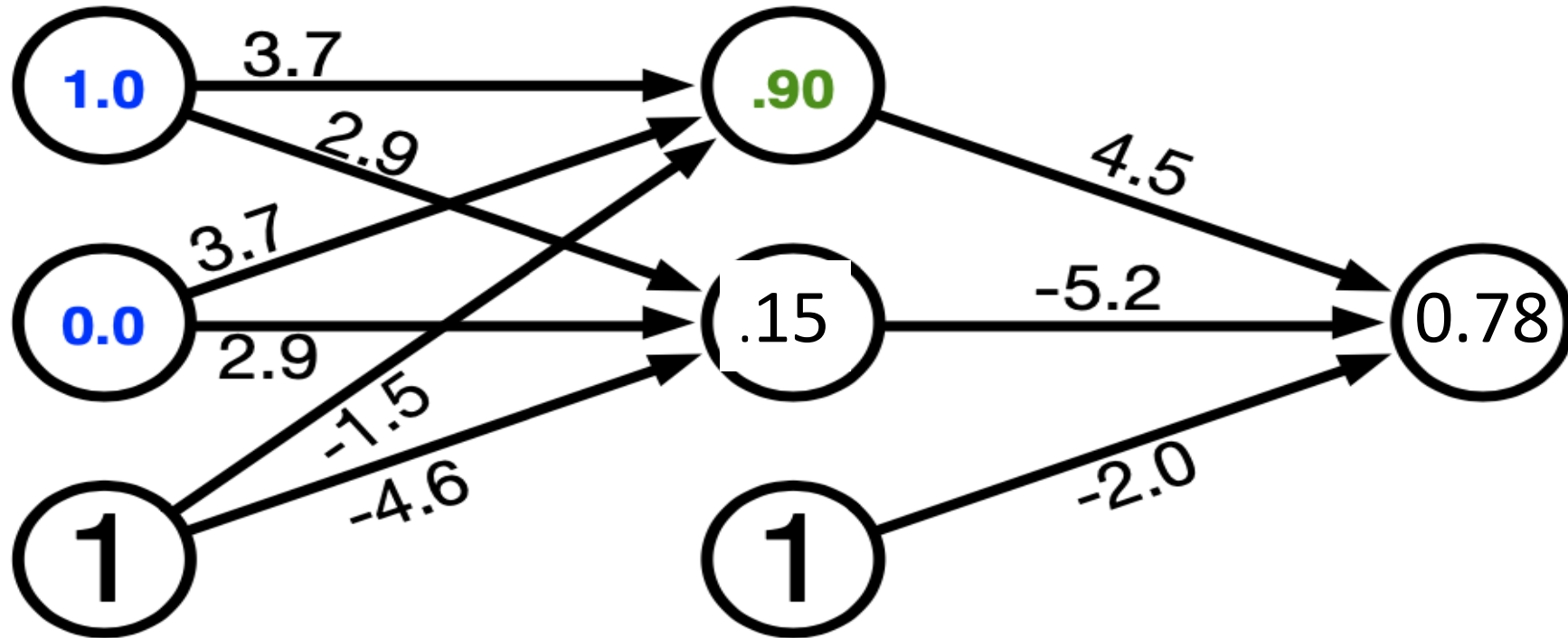
Solution to credit assignment problem in MLP. *Rumelhart, Hinton and Williams (1986)* (though actually invented earlier in a PhD thesis relating to economics, Paul Werbos: Harvard)
BP has two phases:

1. Forward pass: computes 'functional signal', feed forward propagation of input pattern signals through the network.
2. Backward pass: computes 'error signal', *propagates* the error *backwards* through network starting at output units (where the error is the difference between actual and desired/target output values)

Back-Propagation: Forward Pass

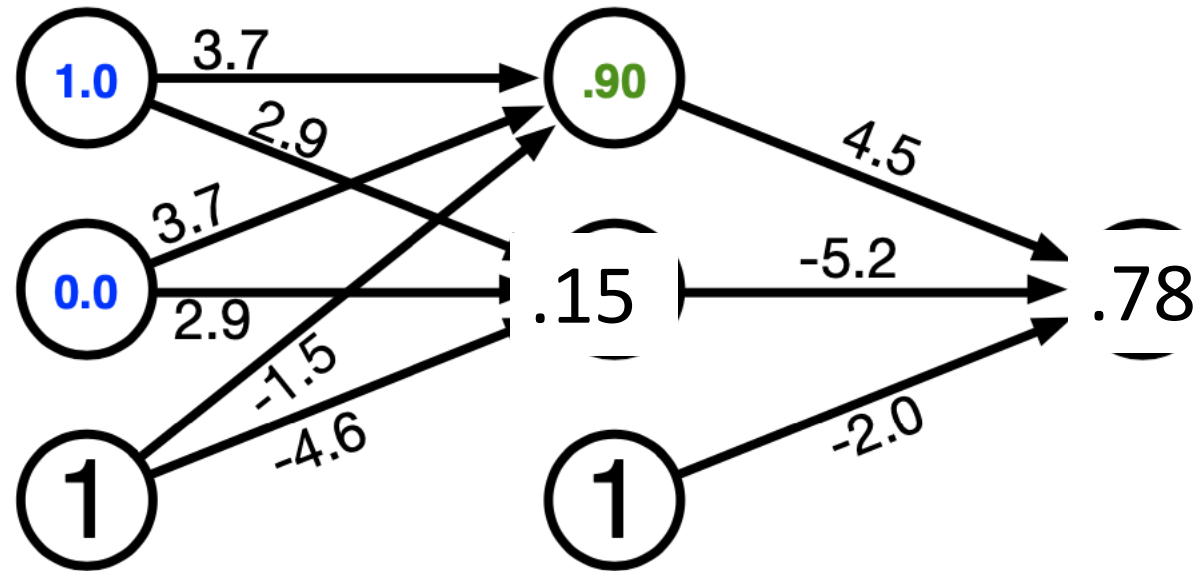
- Step 1: Initialize weights at random, choose a learning rate η
 - Until network is trained:
 - For each training example i.e. input pattern and target output(s):
- Step 2: Do forward pass through the net (with fixed weights) to produce output(s)
 - i.e., in Forward Direction, layer by layer:
 - Inputs applied
 - Multiplied by weights
 - Summed
 - 'Squashed' by sigmoid activation function
 - Output passed to each neuron in next layer
 - Repeat above until network output(s) produced

Output Computation



- Output: $\text{sigmoid}(.90 \times 4.5 + .15 \times -5.2 + 1 \times -2.0) = \text{sigmoid}(1.27) = \frac{1}{1 + e^{-1.17}} = 0.78$
- The forward pass ends here with the output value being 0.78

Identifying Error



- Computed output: $y = 0.78$
- Target output: $t = 1.0$
- Error = $f(|y-t|)$
- How to reduce the error?

Error: Some Observations

- The computed output, y , depends on the weights
- So, the error, $t-y$, is a function of weights even though t is fixed for a given input.
- We update the weights to reduce the error
- We formulate the error as a function of weights and attempt to minimize the error.
- Start with some initial choice of weights and update them to move in the direction of minimizing the error; gradient descent.
- This is achieved with the help of error back-propagation.

Derivative of the Sigmoid Function

1. Sigmoid Function: $Sigmoid(x) = \frac{1}{1+e^{-x}}$

2. Quotient Rule for Differentiation: $\left(\frac{f(x)}{g(x)}\right)' = \frac{g(x)f'(x) - f(x)g'(x)}{g(x)^2}$

3. Derivative of Sigmoid: $\frac{d}{dx} \frac{1}{1+e^{-x}} = \frac{0 \times (1+e^{-x}) - (-e^{-x})}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \left(\frac{e^{-x}}{1+e^{-x}} \right)$

$$= \frac{1}{1+e^{-x}} \left(1 - \frac{1}{1+e^{-x}} \right) = Sigmoid(x)(1 - Sigmoid(x))$$

Use Error to Update the Final Layer First

- Linear Combination of Weights $s = \sum_i w_i h_i$
- Activation output $y = \frac{1}{1+e^{-s}}$
- Error (Euclidean Norm) $E = \frac{1}{2} (t - y)^2$
- Derivative of error with respect to one weight w_k ,

$$\frac{dE}{dw_k} = \frac{dE}{dy} \frac{dy}{ds} \frac{ds}{dw_k}$$

Update a Weight in the Final Layer

- Derivative of error with respect to one weight w_k ,

$$\frac{dE}{dw_k} = \frac{dE}{dy} \frac{dy}{ds} \frac{ds}{dw_k}$$

- $\frac{dE}{dy} = \frac{d}{dy} \frac{1}{2} (t - y)^2 = -(t-y)$
- $\frac{dy}{ds} = \frac{d \text{ Sigmoid}(s)}{ds} = \text{Sigmoid}(s) (1-\text{Sigmoid}(s)) = y(1-y)$
- $\frac{ds}{dw_k} = \frac{d}{dw_k} \sum_k w_k h_k = h_k$

Update a Weight in the Final Layer

- Derivative of error with respect to one weight w_k ,

$$\begin{aligned}\frac{dE}{dw_k} &= \frac{dE}{dy} \frac{dy}{ds} \frac{ds}{dw_k} \\ &= -(t-y) y(1-y) h_k\end{aligned}$$

- $-(t-y)$ is the error component
- $y(1-y)$ is the derivative of sigmoid: y'
- If η is the learning rate, then the update is
- $\Delta w_k = \eta (t - y) y' h_k$

Multiple Output Nodes

- Our example only had one output node
- Practical neural networks may have multiple output nodes
- Error is computed over all i output nodes given by

$$E = \sum_i \frac{1}{2} (t_i - y_i)^2$$

- Weights from $k \rightarrow i$ are adjusted using

$$\Delta w_{i \leftarrow k} = \eta (t_i - y_i) y'_i h_k$$

Weight Updates in Front of Hidden Layers

- In a hidden layer, we do not have a target output value
- But we can compute how much each node contributed to downstream error

- Definition of error term of each output node

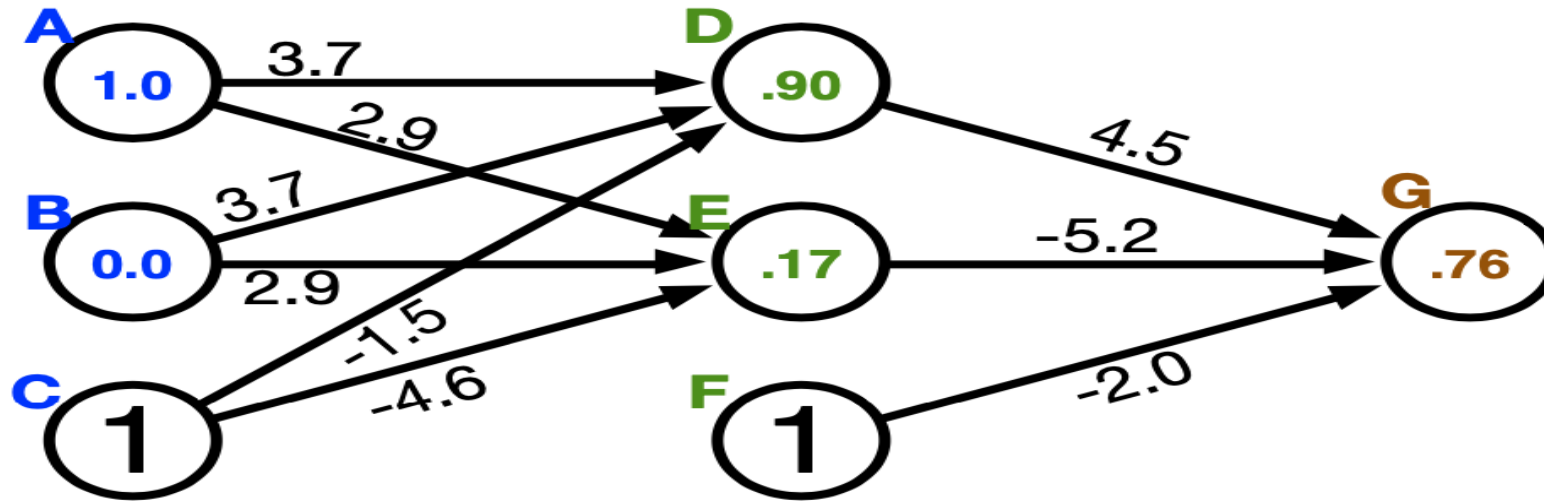
$$\delta_j = (t_j - y_j) y_j'$$

- Back-propagate the error term

$$\delta_i = (\sum_j w_{j \leftarrow i} \delta_j) y_i'$$

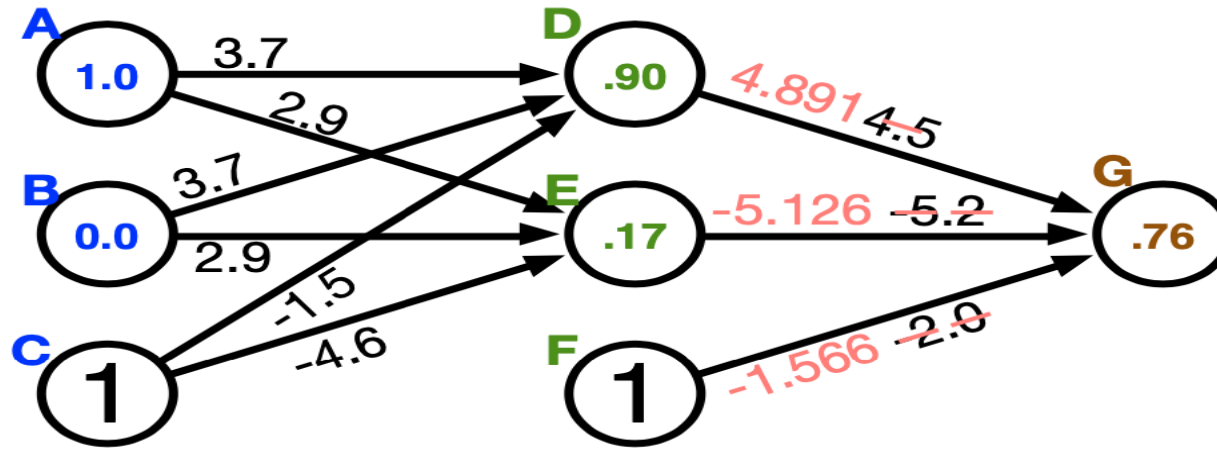
- Universal update formula: $\Delta w_{j \leftarrow k} = \eta \delta_j h_k$

Example: Error Back-Propagation



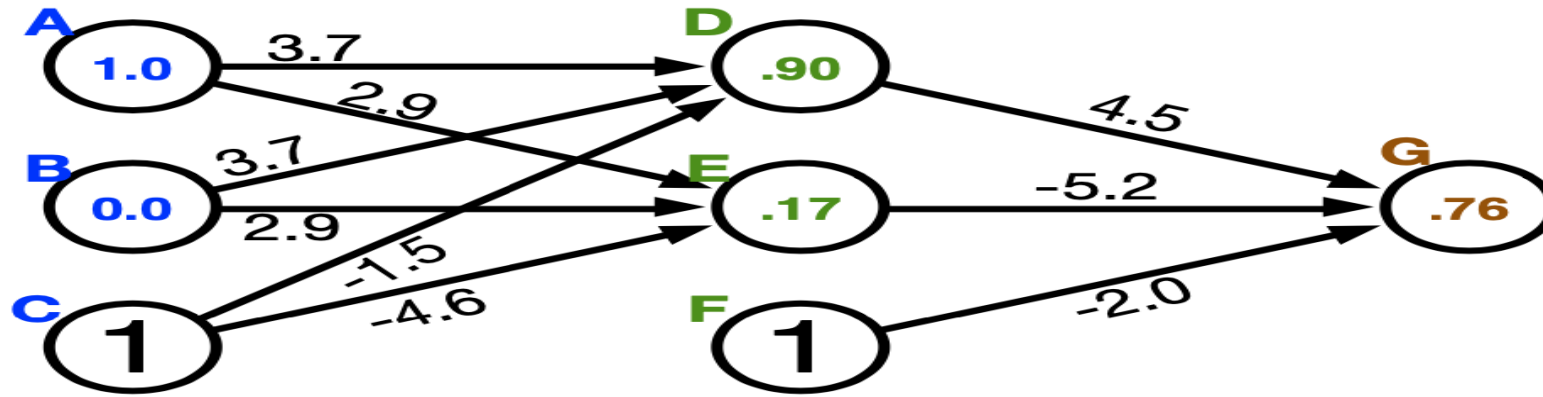
- Computed output: $y = .78$
- Correct output: $t = 1.0$
- Final layer weight updates (learning rate $\eta = 10$)
 - $\delta_G = (t - y) y'_i = (1 - 0.78) 0.17 = .0374$
 - $\Delta w_{GD} = \eta \delta_G h_D = 10 \times .0374 \times .90 = 0.336$
 - $\Delta w_{GE} = \eta \delta_G h_E = 10 \times .0374 \times .17 = 0.064$
 - $\Delta w_{GF} = \eta \delta_G h_F = 10 \times .0374 \times 1 = 0.374$

Example: Error Back-Propagation



- Computed output: $y = .78$
- Correct output: $t = 1.0$
- Final layer weight updates (learning rate $\eta = 10$)
 - $\delta_G = (t-y) y'_i = (1-0.76)0.181 = .0374$
 - $\Delta w_{GD} = \eta \delta_G h_D = 10 \times .0374 \times .90 = 0.336$
 - $\Delta w_{GE} = \eta \delta_G h_E = 10 \times .0374 \times .17 = 0.064$
 - $\Delta w_{GF} = \eta \delta_G h_F = 10 \times .0374 \times 1 = 0.374$

Updating Weights in the Hidden Layers



- Hidden Node D: $\delta_D = (\sum_j w_{j \leftarrow i} \delta_j) y'_D = 4.5 \times 0.0434 \times 0.0898 = 0.0175$
- $\Delta w_{DA} = \eta \delta_D h_A = 10 \times 0.0175 \times 1.0 = 0.175$
- $\Delta w_{DB} = \eta \delta_D h_B = 10 \times 0.0175 \times 0.0 = 0$
- $\Delta w_{DC} = \eta \delta_D h_C = 10 \times 0.0175 \times 1.0 = 0.175$
- Hidden Node E: $\delta_E = (\sum_j w_{j \leftarrow i} \delta_j) y'_E = -5.2 \times 0.0434 \times 0.2055 = -0.0464$
- $\Delta w_{EA} = \eta \delta_E h_A = 10 \times -0.0464 \times 1.0 = -0.464$
- ----

Error Back-Propagation

Step 3: Back-Propagation:

- Compute error (delta or local gradient) for each output unit δ_k
- Layer-by-layer, compute error (delta or local gradient) for each hidden unit δ_j by backpropagating errors (as shown previously)

Step 4: Next, update all the weights Δw_{ij}

By gradient descent, and go back to Step 2

- The overall MLP learning algorithm, involving forward pass and backpropagation of error (until the network training completion), is known as the Generalised Delta Rule (GDR), or more commonly, the Back Propagation (BP) algorithm

'Back-prop' Algorithm Summary

- ◆ Initialise weights at random, choose a learning rate η
- ◆ Until network is trained:
 - ◆ For each training example (input pattern and target outputs):
 - Do forward pass through net (with fixed weights) to produce network outputs
 - For each output unit k , compute deltas (local gradients):
 - For hidden units j (from last to first hidden layer, for the case of more than 1 hidden layer) compute deltas (local gradients):
 - For all weights, change weight by gradient descent (generalized Delta Rule):
$$\Delta w_{jk} = \eta \delta_k o_j$$

where o_j is the input to the neuron k , and η is the learning rate, and δ_k is local gradient for the neuron

Training

- This was a single iteration of back-prop
- Training requires many iterations with many training examples or *epochs* (one epoch is entire presentation of complete training set)
- It can be slow !
- Note that computation in MLP is local (with respect to each neuron)
- Parallel computation implementation is also possible